

IMPLEMENTING SECURITY INTO A SECURE PASSWORD MANAGER APPLICATION

Executive summary

Project objective. This project set out to harden a self-hosted password-manager deployment — Vaultwarden, an unofficial Bitwarden-compatible server — running on Docker within a controlled Kali Linux laboratory, and to verify the resulting security posture through direct system inspection rather than through vendor claims alone.

Problem addressed. Password managers concentrate a large amount of sensitive credential material behind a single master secret. A self-hosted deployment removes reliance on a third-party operator, but shifts responsibility for transport security, encryption configuration, access control, backup integrity, and monitoring onto the person operating the server. The project treats each of these as a requirement to be configured and then independently verified.

Solution implemented. Vaultwarden was deployed via Docker Compose, initially over plain HTTP on port 8000 and subsequently reconfigured to serve over HTTPS on port 8443 using Rocket's built-in TLS support with a self-signed certificate. Argon2id was configured as the key-derivation function, two-step login (TOTP) was enabled, an organization with a collection and a second, restricted member account was created to exercise role-based access control, and both database-only and full (database plus RSA key) backups were created, encrypted with OpenSSL, and restored to verify integrity.

Security controls implemented. HTTPS/TLS with a full set of security response headers (HSTS, CSP, X-Frame-Options, Permissions-Policy and others); Argon2id key derivation; client-side field-level encryption (Bitwarden CipherString format, AES-256-CBC with HMAC-SHA256); TOTP-based two-step login; an organization/collection structure with a restricted member role; AES-256-CBC/PBKDF2-encrypted backups with a tested recovery path; and structured application logging capturing authentication and vault-operation events.

Testing performed. Testing was carried out directly against the running container and its data using standard Linux and cryptographic tooling: curl and openssl for HTTPS/TLS verification; sqlite3 for schema and ciphertext inspection; openssl enc/dec plus sha256sum and PRAGMA integrity_check for backup and recovery verification; the Vaultwarden web UI and a second invited account for authentication, MFA, and RBAC testing; and docker logs / docker inspect / docker stats for monitoring evidence.

Major findings. The core cryptographic and access-control requirements were implemented and are verifiable directly in the evidence: ciphertext in the database consistently uses the Bitwarden "2." CipherString prefix, backups decrypt to byte-identical, integrity-checked databases, and a restricted member account was observed with access limited to its assigned collection. Two authentication-failure paths were captured in the server logs: an invalid master password and a missing TOTP token, confirming that both factors are independently enforced.

Major limitations. Some items could not be verified in the available environment: no mobile client or browser-extension autofill testing was performed (the extension was never installed), and features such as emergency access, SSO, Duo, and Bitwarden Send are present in the schema but were not exercised. Two configuration issues were also surfaced by the evidence itself rather than resolved by it: Vaultwarden repeatedly logged a notice that the ADMIN_TOKEN was stored in plain text rather than as an Argon2 hash, and the rsa_key.pem file was observed with two different permission states (644 and 600) at different points in testing, so its final state cannot be asserted with confidence.

Overall outcome. The deployment meets its core objectives for encryption, authentication, access control, and backup integrity, each independently verified rather than assumed. It is best characterised as a working laboratory proof of concept rather than a production-ready deployment: the recommendations in Section 16 identify the specific hardening steps — principally a trusted certificate, a hashed admin token, and disabling open sign-ups — needed before any production use.

1. Introduction

1.1 Background

Password managers are widely recommended as a defence against credential reuse and weak-password selection, but they also create a high-value target: a single master secret protects every credential the user has stored. Self-hosting a password manager - as opposed to using a commercial cloud service - gives the operator direct control over the server, the data at rest, and the network path, but it also means every one of those layers has to be configured and verified by the operator rather than by a vendor's security team.

Vaultwarden is an unofficial, community-maintained server implementation of the Bitwarden API, written in Rust, that is compatible with the official Bitwarden client applications (web vault, browser extension, desktop, and mobile) while being significantly lighter to self-host than Bitwarden's own reference server. Because it re-implements the Bitwarden protocol rather than being endorsed by Bitwarden, its security properties benefit from independent verification rather than being taken on trust.

1.2 Problem statement

A self-hosted password manager is only as secure as its weakest configured layer: transport encryption, vault-level cryptography, authentication strength, access control, backup handling, and monitoring. Default or partially-configured deployments can leave one or more of these layers weak even where others are strong. This project addresses the question: when Vaultwarden is deployed and then deliberately hardened across these layers, which controls can be shown to actually work, and which limitations remain, when tested directly rather than assumed from documentation?

1.3 Project aim

To deploy Vaultwarden in a controlled laboratory environment, apply a defined set of security controls (encryption, authentication, access control, backup, monitoring), and verify each control through direct, reproducible technical testing, producing an evidence-backed account of what was achieved and what was not.

1.4 Project objectives

- Deploy Vaultwarden using Docker and Docker Compose on Kali Linux.
- Configure and verify HTTPS/TLS transport protection.
- Configure and verify Argon2id key derivation and inspect the resulting vault ciphertext format.
- Enable and test TOTP-based multi-factor authentication, including a failure path.
- Create an organization/collection structure and verify role-based access restrictions with a second account.
- Create, encrypt, and successfully restore both database-only and full backups.
- Exercise core usability features (search, password generator availability, weak-password reporting).
- Capture and review application logs for security-relevant events.
- Identify limitations and produce hardening recommendations for a future production deployment.

1.5 Scope

The scope of this project covers a single-node Vaultwarden deployment inside Docker on a Kali Linux virtual machine, accessed over the local loopback interface (127.0.0.1) and the host's local network adapter. Testing covers the web vault client, the Vaultwarden REST/API layer, the SQLite data store, and the OpenSSL-based backup process.

1.6 Out of scope

- Mobile client (iOS/Android) installation and testing.
- Browser-extension installation and autofill testing.
- Production-scale deployment, external DNS, or a certificate issued by a public certificate authority.
- Penetration testing of the underlying Kali Linux host, Docker engine, or network stack.
- Load, performance, or scalability testing.

1.7 Assumptions

- The Kali Linux host itself (OS patch level, kernel hardening) is trusted for the purposes of this project; host-level compromise is discussed only in the threat model (Section 14) as a residual risk.
- The self-signed TLS certificate is acceptable for a laboratory environment and is not intended to represent a production trust model.
- "Not verified in the controlled laboratory environment" is used throughout this report to describe capabilities that Vaultwarden documents or supports architecturally but for which no test evidence was captured during this project.

1.8 Controlled laboratory environment

All testing was performed on a single Kali GNU/Linux Rolling (2026.3) virtual machine, user account "ali", with Vaultwarden running as a Docker container reachable at 127.0.0.1:8443 (and, in the initial deployment stage, 127.0.0.1:8000 over plain HTTP before TLS was configured). No public-internet exposure, reverse proxy, or externally trusted certificate was used at any point; this is reflected consistently throughout the report and is treated as a laboratory constraint rather than a failure.

2. Technology and environment

The deployment stack was verified directly from the running system rather than assumed from the project brief. The table below lists each component together with the specific version observed in the evidence.

Table 2.1 — Verified technology stack

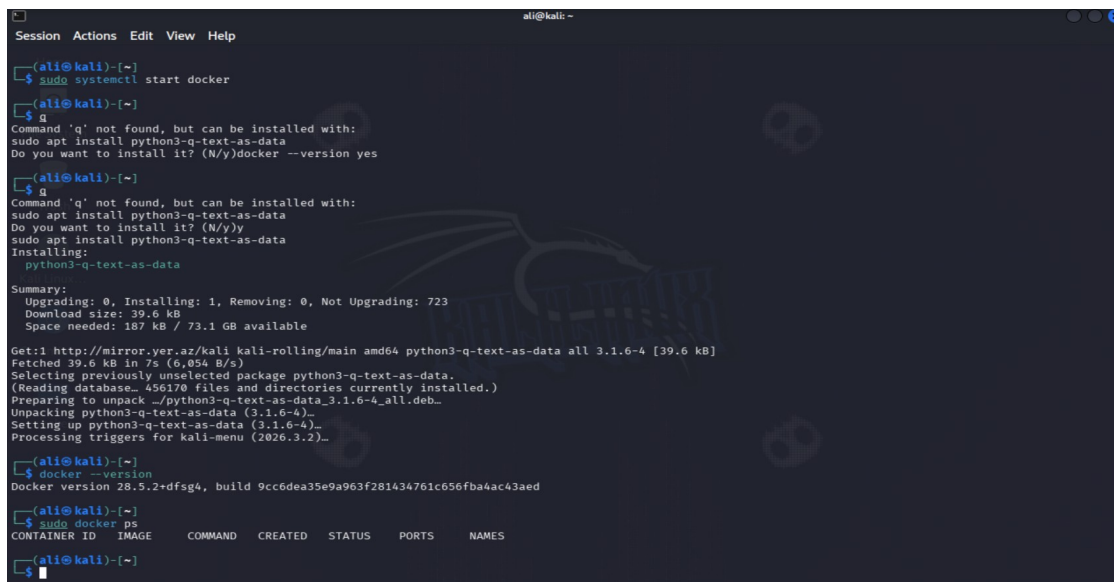
Component	Verified version / detail
Host OS	Kali GNU/Linux Rolling 2026.3 (kernel 7.0.12-2kali1)
Container engine	Docker 28.5.2 (build 9cc6dea)
Orchestration	Docker Compose 2.40.3
Application	Vaultwarden 1.37.2 (unofficial Bitwarden-compatible server), Web-Vault 2026.7.0
Data store	SQLite 3.46.1 (file-based, bind-mounted from the host)
Transport security	HTTPS via Rocket's built-in TLS (ROCKET_TLS), self-signed certificate
Backup tooling	OpenSSL 3.6.3 (AES-256-CBC, PBKDF2, 200,000 iterations)

Deployment timeline

Timestamps embedded in the evidence (terminal output, file modification times, and server log entries) allow the deployment to be reconstructed as a short iterative hardening exercise rather than a single one-shot configuration:

- Initial deployment (10 Sep): Vaultwarden started via docker-compose up -d over plain HTTP on port 8000 (DOMAIN=http://127.0.0.1:8000). The container auto-generated its RSA key pair (rsa_key.pem) on this first start.

- TLS hardening (11 Sep): configuration updated to ROCKET_TLS with a certificate and key mounted read-only at /ssl, DOMAIN changed to https://127.0.0.1:8443, and Argon2id key-derivation parameters configured. Database and full backups were created, encrypted, and restore-tested the same day.
- Access-control testing (13-14 Sep): an organization and collection were created, a second member account was invited and used to demonstrate restricted access, cipher-sharing operations were performed, and authentication events - including a rejected master password and a rejected TOTP attempt - were captured in the logs.
- Verification pass (16 Sep): a final scripted HTTPS/TLS and container-health check was run to confirm the hardened configuration remained in place.



```

ali@kali: ~
Session Actions Edit View Help

ali@kali)~)
$ sudo systemctl start docker
ali@kali)~)
$ q
Command 'q' not found, but can be installed with:
sudo apt install python3-q-text-as-data
Do you want to install it? (N/y)docker --version yes
ali@kali)~)
$ q
Command 'q' not found, but can be installed with:
sudo apt install python3-q-text-as-data
Do you want to install it? (N/y)y
sudo apt install python3-q-text-as-data
Installing:
python3-q-text-as-data

Summary:
Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 723
Download size: 39.6 kB
Space needed: 187 kB / 73.1 GB available

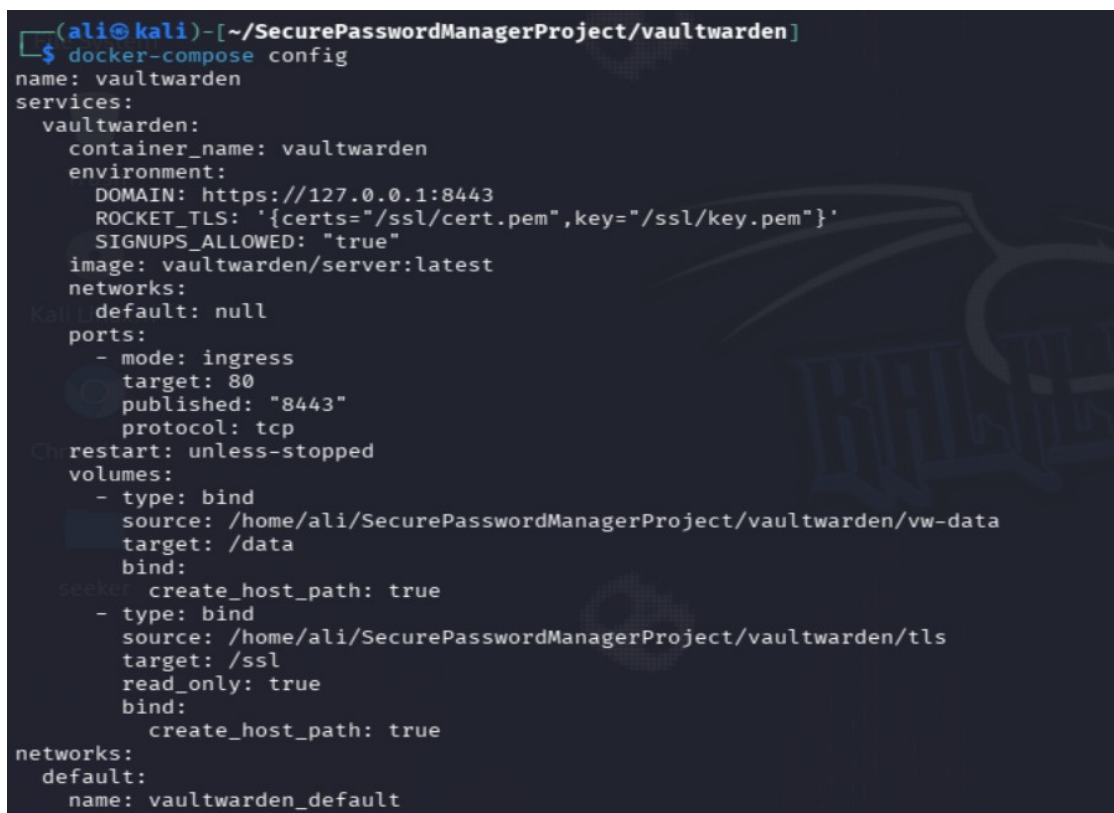
Get:1 http://mirror.yer.az/kali kali-rolling/main amd64 python3-q-text-as-data all 3.1.6-4 [39.6 kB]
Fetched 39.6 kB in 7s (6,054 B/s)
Selecting previously unselected package python3-q-text-as-data.
(Reading database... 456170 files and directories currently installed.)
Preparing to unpack ../python3-q-text-as-data_3.1.6-4_all.deb...
Unpacking python3-q-text-as-data (3.1.6-4)...
Setting up python3-q-text-as-data (3.1.6-4)...
Processing triggers for kali-menu (2020.3.2)...

ali@kali)~)
$ docker --version
Docker version 20.5.2+dfsg4, build 9cc6dea35e9a963f281434761c656fba4ac43aed

ali@kali)~)
$ sudo docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS              NAMES
ali@kali)~)
$

```

Figure 2.1 — Docker installation and initial verification on Kali Linux (docker --version, docker ps).



```

ali@kali)~)
$ docker-compose config
name: vaultwarden
services:
  vaultwarden:
    container_name: vaultwarden
    environment:
      DOMAIN: https://127.0.0.1:8443
      ROCKET_TLS: '{certs="/ssl/cert.pem",key="/ssl/key.pem"}'
      SIGNUPS_ALLOWED: "true"
    image: vaultwarden/server:latest
    networks:
      default: null
    ports:
      - mode: ingress
        target: 80
        published: "8443"
        protocol: tcp
    restart: unless-stopped
    volumes:
      - type: bind
        source: /home/ali/SecurePasswordManagerProject/vaultwarden/vw-data
        target: /data
        bind:
          create_host_path: true
      - type: bind
        source: /home/ali/SecurePasswordManagerProject/vaultwarden/tls
        target: /ssl
        read_only: true
        bind:
          create_host_path: true
    networks:
      default:
        name: vaultwarden_default

```

Figure 2.2 — Final, hardened docker-compose configuration: HTTPS domain, ROCKET_TLS, and a read-only TLS volume mount.

3. Security requirements

The table below maps each project requirement to what was actually implemented and verified in the evidence. Status follows four levels used consistently throughout this report: Verified (implemented and directly confirmed by evidence), Partially verified (implemented but only partly exercised), Documented capability (supported by Vaultwarden's architecture but not exercised in this environment), and Not tested (no evidence captured).

Table 3.1 — Security requirements traceability

Requirement	Implementation	Verification	Status
Strong encryption for stored passwords	Client-side AES-256-CBC + HMAC-SHA256 (Bitwarden CipherString v2)	SQL inspection of ciphers.data / name / username / URI fields	Verified
Secure key derivation	Argon2id, memory 32MB, iterations 6, parallelism 4	Vaultwarden Security > Keys UI	Verified
Password / data integrity	SQLite PRAGMA integrity_check; SHA-256 comparison on backups	Backup and restore tests	Verified
Multi-factor authentication	TOTP via authenticator app	Two-step login settings; login flow; auth-event logs (success, missing-token failure)	Verified
Role-based access control	Organization + collection + member role ("User")	Admin Console Members/Collections; second-account login test	Verified
Secure encrypted backups	OpenSSL AES-256-CBC, PBKDF2, 200,000 iterations	file(1) confirms encrypted format; decrypt + hash + integrity check	Verified
Backup / recovery testing	DB-only and full (DB + RSA key) restore tested	sha256sum match, PRAGMA integrity_check = ok	Verified
Cross-platform client architecture	Bitwarden-compatible API supports web, desktop, mobile, extension	Web vault tested; other clients not installed	Partially verified
Browser integration / autofill	Browser extension offered in vault UI	Extension not installed in this environment	Not tested
Password sharing	Organization collections; cipher share API	PUT /api/ciphers/{id}/share -> 200 OK observed in logs	Verified (via server log, not UI screenshot)
Password health / security checks	Org Reports > Weak passwords	1 item flagged "Very Weak" in Reports UI	Verified
Monitoring and logging	Rocket structured request/response + auth-event logging	docker logs, docker inspect, docker stats	Verified
Security testing	HTTPS headers, TLS cert, auth failure paths, backup integrity	curl, openssl, sqlite3, log analysis	Verified
User training / documentation	Not part of the supplied evidence set	No training material submitted	Not tested

4. Security architecture

Vaultwarden runs as a single Docker container built from the vaultwarden/server:latest image (1.37.2). The Rocket web framework inside the container terminates TLS directly - there is no separate reverse proxy in this deployment - and serves both the API and the compiled web-vault client. Vault data is persisted outside the container through a bind mount, and a second, read-only bind mount supplies the TLS certificate and key.

It is important to be precise about what the server can and cannot see: Vaultwarden's server-side role is authentication, authorization, and ciphertext storage. Vault contents are encrypted and decrypted entirely on the client; the server never has access to a user's master password or to plaintext vault data. The diagram below reflects this - the "client-side vault encryption" stage sits conceptually on the client side of the trust boundary even though, for compactness, it is drawn alongside the server-side authentication and authorization stages it interacts with.

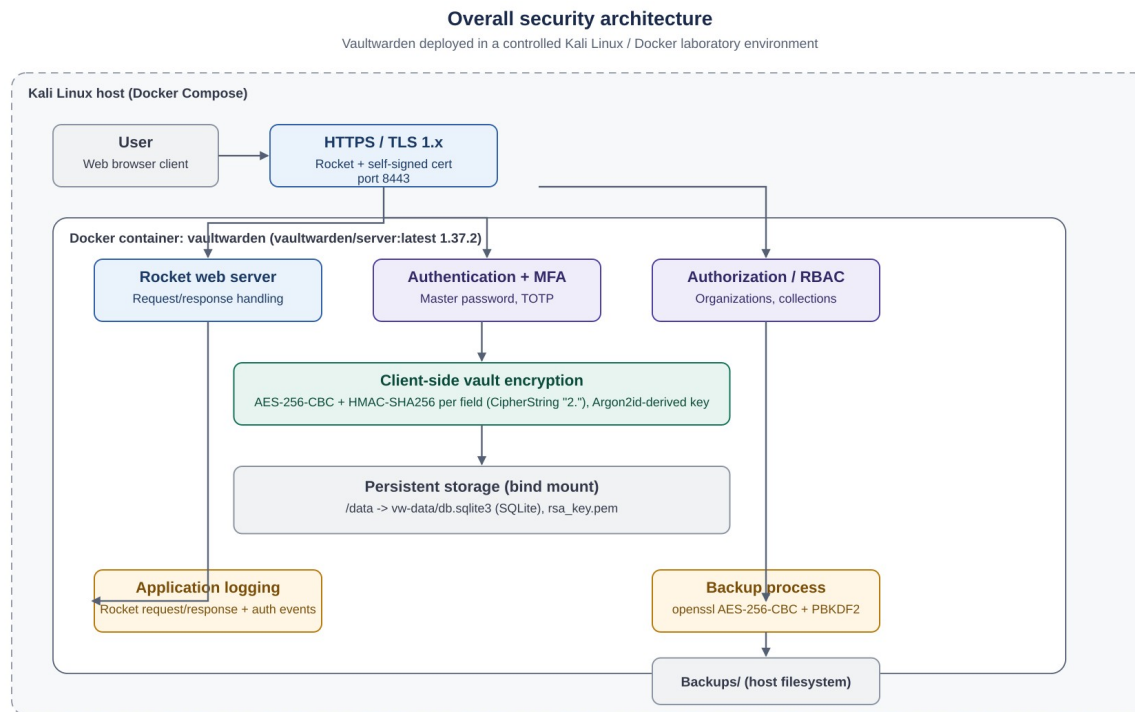


Figure 4.1 — Overall security architecture of the lab deployment.

4.1 Container and network configuration

- Container: vaultwarden, image vaultwarden/server:latest, running as `"/start.sh"`.
- Port mapping: 8443 (host) -> 80 (container), TCP, on both IPv4 and IPv6.
- Volumes: vw-data (host) -> /data (container), read-write; tls (host) -> /ssl (container), read-only.
- Environment: `DOMAIN=https://127.0.0.1:8443`,
`ROCKET_TLS={certs="/ssl/cert.pem",key="/ssl/key.pem"}`, `SIGNUPS_ALLOWED=true`.

```

ali@kali:~/SecurePasswordManagerProject/vaultwarden$ docker exec vaultwarden /vaultwarden --version
Vaultwarden 1.37.2
Web-Vault 2026.7.0

ali@kali:~/SecurePasswordManagerProject/vaultwarden$ docker inspect vaultwarden --format='{{.json .Mounts}}'
[{"Type":"bind","Source":"/home/ali/SecurePasswordManagerProject/vaultwarden/vw-data","Destination":"/data","Mode":"rw","RW":true,"Propagation":"rprivate"}, {"Type":"bind","Source":"/home/ali/SecurePasswordManagerProject/vaultwarden/tls","Destination":"/ssl","Mode":"ro","RW":false,"Propagation":"rprivate"}]
  
```

Figure 4.2 — docker inspect output confirming the data and TLS bind mounts and their access modes.

4.2 Deployment evolution

The evidence shows the deployment was not hardened in a single step. The container was first brought up with docker-compose up -d on plain HTTP (port 8000), then reconfigured to the HTTPS/TLS configuration shown above. Both stages are represented in the evidence set and are treated in this report as sequential states of the same project rather than as separate deployments.

```

ali@kali: ~/SecurePasswordManagerProject/vaultwarden
Session Actions Edit View Help
Processing triggers for kali-menu (2026.3.2)...
Processing triggers for man-db (2.13.1-1)...

ali@kali) ~/SecurePasswordManagerProject/vaultwarden/vw-data
$ docker-compose --version
Docker Compose version 2.40.3-3

ali@kali) ~/SecurePasswordManagerProject/vaultwarden/vw-data
$ cd ..

ali@kali) ~/SecurePasswordManagerProject/vaultwarden
$ pwd
/home/ali/SecurePasswordManagerProject/vaultwarden

ali@kali) ~/SecurePasswordManagerProject/vaultwarden
$ ls -la
total 12
drwxrwxr-x 3 ali ali 4096 Sep 10 23:38 .
drwxrwxr-x 19 ali docker 4096 Sep 10 23:37 ..
drwxrwxr-x 2 ali ali 4096 Sep 10 23:38 vw-data

ali@kali) ~/SecurePasswordManagerProject/vaultwarden
$ nano compose.yml

ali@kali) ~/SecurePasswordManagerProject/vaultwarden
$ cat compose.yml
services:
  vaultwarden:
    image: vaultwarden/server:latest
    container_name: vaultwarden
    restart: unless-stopped

    environment:
      DOMAIN: "http://127.0.0.1:8000"

    volumes:
      - ../vw-data:/data

    ports:
      - "8000:80"

```

Figure 4.3 — Initial plain-HTTP configuration (port 8000), superseded by the HTTPS configuration shown in Section 2.

4.3 Scope note: SIGNUPS_ALLOWED

Observation

The final configuration retains `SIGNUPS_ALLOWED=true`, meaning the server would accept new self-registrations if it were reachable beyond the loopback interface. This is appropriate for a single-operator lab test but is flagged as a production hardening item in Section 16.

5. Encryption and key management

Bitwarden-compatible clients (and therefore Vaultwarden) use an envelope-encryption model: a master password, combined with the account email as salt, is stretched through a key-derivation function to produce a master key, which in turn protects a randomly generated account symmetric key. That symmetric key is what actually encrypts vault items, field by field, on the client before anything is sent to the server. This project verified the key-derivation configuration and the resulting ciphertext format directly; it did not have access to the client-side key-derivation or encryption source code execution itself, since that happens inside the browser, outside the scope of server-side inspection.

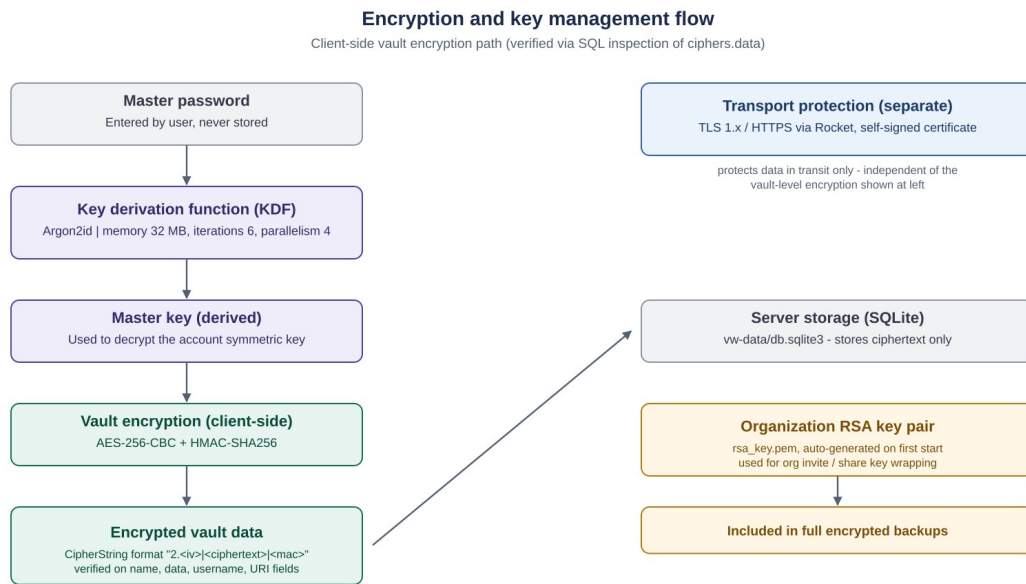


Figure 5.1 — Encryption and key-management flow, annotated with the parameters and formats verified in this project.

5.1 Key derivation (Argon2id)

The account's key-derivation function was configured in the Vaultwarden web UI under Settings > Security > Keys as: Argon2id, memory 32 MB, 6 iterations, parallelism 4. These are stronger than Bitwarden's historical PBKDF2 default and are in line with OWASP's current password-hashing guidance for Argon2id.

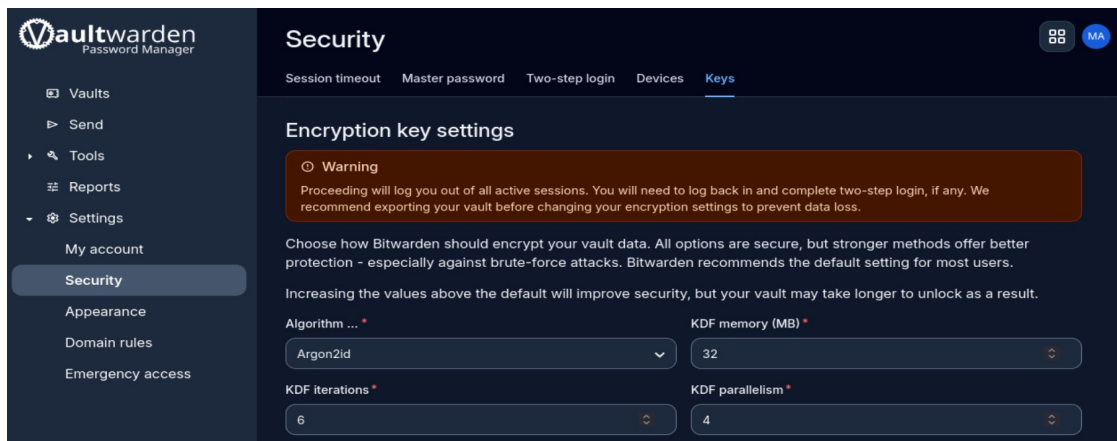


Figure 5.2 — Argon2id KDF configuration observed in the Vaultwarden web vault.

5.2 Verified ciphertext format

Direct SQL queries against `vw-data/db.sqlite3` confirm that the `name`, `data` (which itself contains encrypted `username`, `uri`, and `uriChecksum` sub-fields), and related columns of the `ciphers` table are stored as Bitwarden `CipherString` values beginning with the prefix `"2."` - the format Bitwarden defines for AES-256-CBC encryption with an HMAC-SHA256 authentication tag, base64-encoded and pipe-delimited (`IV | ciphertext | MAC`). No plaintext vault content was observed in the database at rest.

```
(ali@kali)~[/SecurePasswordManagerProject/vaultwarden]
$ sqlite3 vw-data/db.sqlite3 "SELECT uuid, substr(name,1,80) AS encrypted_name FROM ciphers;"
521a64e0-439a-483b-aa7a-000d02c5de1e|2.5UXQg+tDu+9oPQos0UBtGA==H2f0REQR5qglT5Pix5G1bAWDzrS8QdyFygP3ZpvIbo8==eb+XAwgd
18fa87a3-9109-4798-b9fd-05c134fe905e|2.NndDPm2550YqKX0zK6j3Xg==nL/h1io+dRdSlxQakE/xvg==7Fo/sjyTE+Bj1HPbGca4j0BlqND

(ali@kali)~[/SecurePasswordManagerProject/vaultwarden]
$ docker exec vaultwarden /vaultwarden --version
Vaultwarden 1.37.2
Web-Vault 2026.7.0

(ali@kali)~[/SecurePasswordManagerProject/vaultwarden]
$ docker exec vaultwarden env | grep -E 'DOMAIN|ROCKET_TLS|SIGNUPS_ALLOWED'
ROCKET_TLS={certs="/ssl/cert.pem",key="/ssl/key.pem"}
SIGNUPS_ALLOWED=true
DOMAIN=https://127.0.0.1:8443
```

Figure 5.3 — SQL query on `ciphers.name` showing the "2." CipherString prefix, alongside Vaultwarden version and TLS environment confirmation.

```
(ali@kali)~[/SecurePasswordManagerProject/vaultwarden]
$ sqlite3 vw-data/db.sqlite3 "SELECT uuid, name, substr(name,1,3) AS name_prefix, substr(data,1,3) AS data_prefix FROM ciphers;"
521a64e0-439a-483b-aa7a-000d02c5de1e|2.5UXQg+tDu+9oPQos0UBtGA==H2f0REQR5qglT5Pix5G1bAWDzrS8QdyFygP3ZpvIbo8==eb+XAwgdvRkIoSN6t8D9hDy1g5LYkyGhbD4oFAEDM0o==2.5l{"t
18fa87a3-9109-4798-b9fd-05c134fe905e|2.NndDPm2550YqKX0zK6j3Xg==nL/h1io+dRdSlxQakE/xvg==7Fo/sjyTE+Bj1HPbGca4j0BlqND9RahmaI+DqrtIfA==2.Nl{"t

(ali@kali)~[/SecurePasswordManagerProject/vaultwarden]
$ sqlite3 vw-data/db.sqlite3 "SELECT uuid, substr(data,3,120) FROM ciphers;"
521a64e0-439a-483b-aa7a-000d02c5de1e|{"totp":null,"uris":["uriChecksum":"2.UFUSLa/abtizrVaIPDorng==fER7ZCHcvIoZ27dd+9HX72cCLOToA6WEXQsofY+mRpsbpvCTsvlM5e
7K
18fa87a3-9109-4798-b9fd-05c134fe905e|{"totp":null,"uris":[],"username":"2.WFpuwN8rhqrqCIE0xykGQ==BDBggj1a10FCTGn+Ut27OKK2Gep7G8AFpawm+y2YD9k==UXxuAWlry6B
Dl
```

Figure 5.4 — Field-level confirmation: name, data, username, and URI values all carry the "2." CipherString prefix.

5.3 Schema and per-item keys

The `ciphers` table schema includes an optional per-item key column, used by Bitwarden clients to re-key individual items independently of the account key. In the two cipher records present in the test dataset, this column was NULL for both ($\text{COUNT}(\text{key}) = 0$ out of $\text{COUNT}(\ast) = 2$), meaning both items rely on the account-level encryption key rather than an item-specific key. This is normal default behaviour and not a defect, but it is recorded here as an observed fact rather than assumed.

```
(ali@kali)~[/SecurePasswordManagerProject/vaultwarden]
$ cd ~/SecurePasswordManagerProject/vaultwarden

(ali@kali)~[/SecurePasswordManagerProject/vaultwarden]
$ sqlite3 vw-data/db.sqlite3 "PRAGMA table_info(ciphers);";
0|uuid|TEXT|1||1
1|created_at|DATETIME|1||0
2|updated_at|DATETIME|1||0
3|user_uuid|TEXT|0||0
4|organization_uuid|TEXT|0||0
5|atype|INTEGER|1||0
6|name|TEXT|1||0
7|notes|TEXT|0||0
8|fields|TEXT|0||0
9|data|TEXT|1||0
10|password_history|TEXT|0||0
11|deleted_at|DATETIME|0||0
12|reprompt|INTEGER|0||0
13|key|TEXT|0||0

(ali@kali)~[/SecurePasswordManagerProject/vaultwarden]
$ sqlite3 vw-data/db.sqlite3 "SELECT COUNT(*) AS total_ciphers, COUNT(key) AS records_with_key FROM ciphers;"
2|0
```

Figure 5.5 — `ciphers` table schema and a query confirming zero per-item keys in the current dataset.

5.4 Transport protection is a separate layer

HTTPS/TLS (Section 6 covers the specific test evidence) protects data while it moves between the browser and the server, but it is architecturally independent of the vault-level field encryption described above. Disabling TLS would not expose vault contents in plaintext, because the browser already encrypts each field before the request is sent; conversely, TLS alone would not protect vault contents if the server-side ciphertext were somehow weaker, which is why both layers were tested separately in this project.

5.5 Organization RSA key pair

Vaultwarden automatically generated an RSA key pair (`rsa_key.pem`, 1,675 bytes) on first start; this key is used by the Bitwarden protocol to wrap organization and sharing keys when items are shared into a collection. Its file permissions are discussed in Section 8.4, since the evidence shows two different permission states at different points in testing.

Scope note

Exact AES/HMAC parameters below the CipherString-prefix level (e.g. IV size, MAC construction) were not independently re-derived by hand in this project; they are stated here on the basis of the observed "2." prefix together with Vaultwarden/Bitwarden's published CipherString specification, not from decrypting a cipher end-to-end.

6. Authentication and MFA

Authentication in this deployment requires two independently-verified factors before vault access is granted: the account master password, and a time-based one-time password (TOTP) from an authenticator app. Both the success path and two distinct failure paths are captured directly in Vaultwarden's own request/response and authentication-event logs, which is stronger evidence than screenshots of the UI alone.

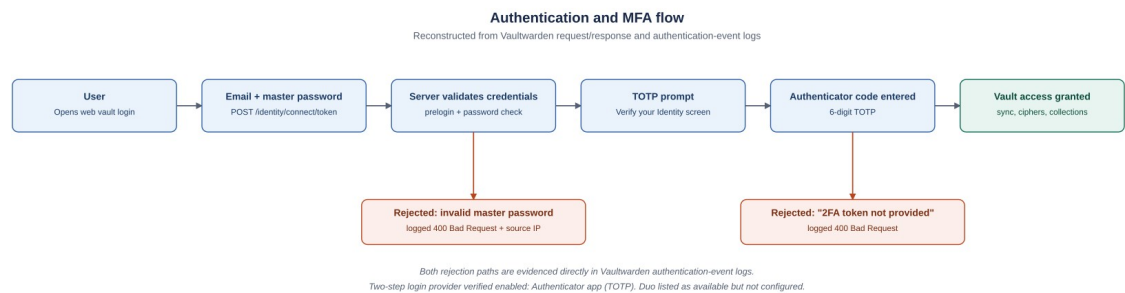


Figure 6.1 — Authentication and MFA flow, including both rejection paths observed in the server logs.

6.1 Two-step login configuration

Settings > Security > Two-step login shows the Authenticator app provider enabled (green check). Duo is listed as an available provider but was not configured - it appears in the UI purely because Vaultwarden advertises it as a supported method, not because it was set up or tested.

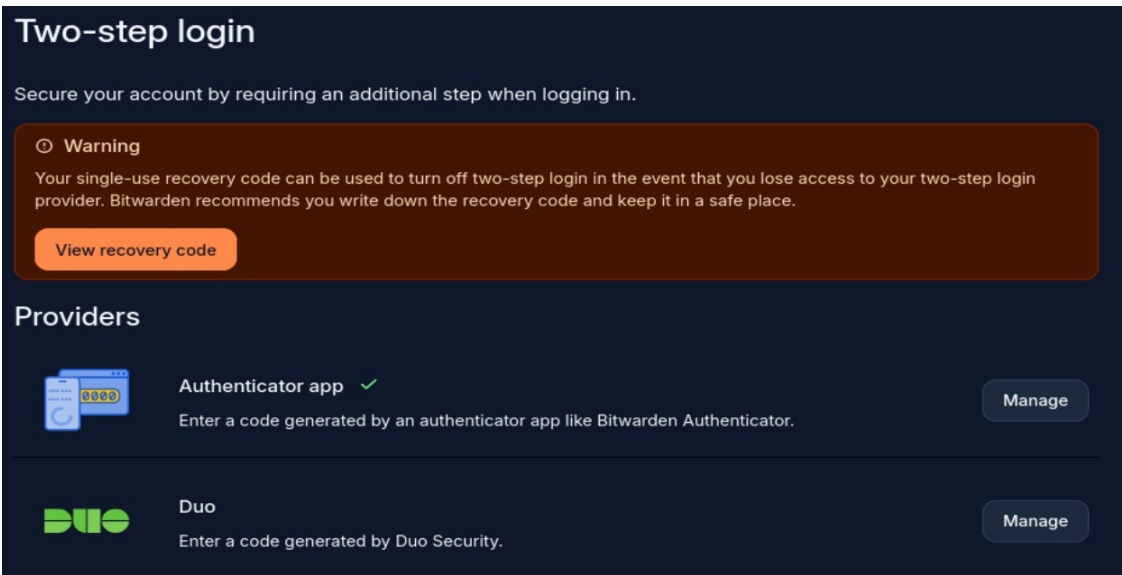


Figure 6.2 — Two-step login settings: Authenticator app (TOTP) enabled; Duo available but not configured.

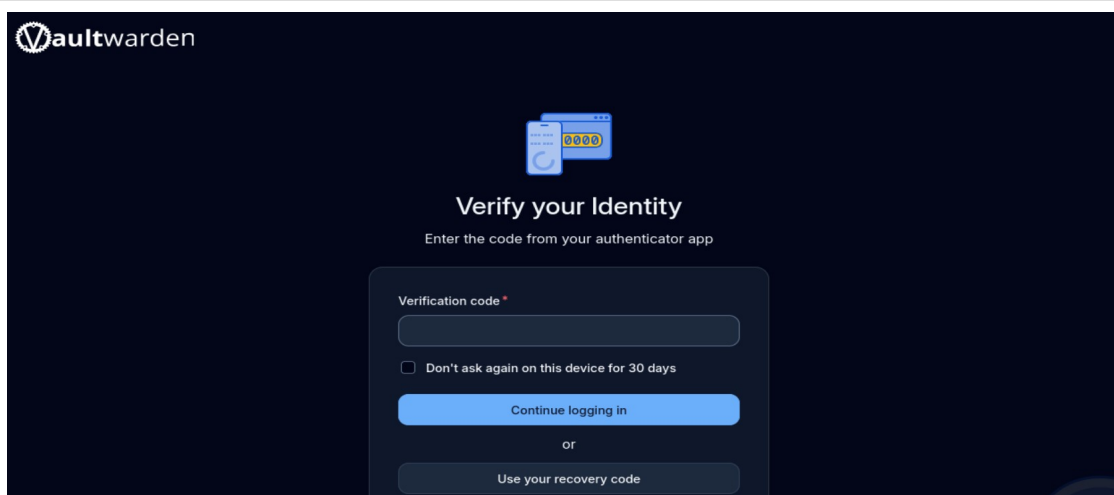


Figure 6.3 — TOTP verification prompt presented at login, with a recovery-code fallback option visible.

6.2 Verified failure paths

Two separate authentication rejections appear in the evidence, and it is important to distinguish them precisely rather than conflate them:

- Invalid master password: the login UI displayed "Invalid master password. Confirm your email is correct and your account was created on 127.0.0.1:8443", and the corresponding server log recorded an ERROR-level entry — "Username or password is incorrect" — together with the source IP and account email, resulting in a 400 Bad Request response to POST /identity/connect/token.
- Missing TOTP token: a separate log entry recorded "[error][ERROR] 2FA token not provided", also resulting in a 400 Bad Request on the same endpoint. This is the actual evidence of an MFA-specific rejection.

Evidence correction

One supplied screenshot is named "MFA_Failed_Login_Test.png", but its content shows the invalid-master-password rejection above, not a rejected TOTP code. The genuine MFA-failure evidence ("2FA token not provided") appears in a different file. Both are reported accurately here rather than trusting the original filename.

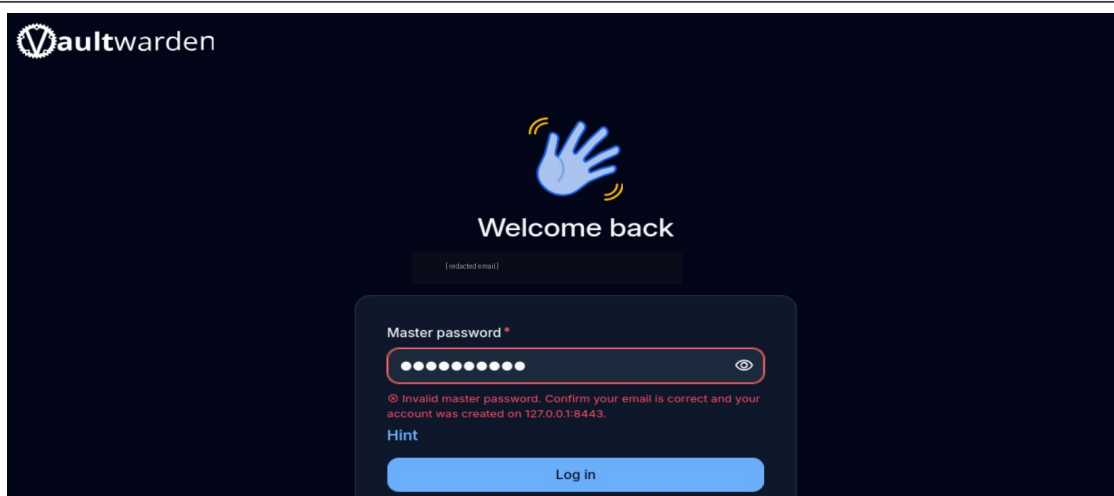


Figure 6.4 — Invalid master password rejection at the login screen (email redacted).

```

[2026-09-14 12:55:56.962][error][ERROR] 2FA token not provided
[2026-09-14 12:55:56.943][response][INFO] (login) POST /identity/connect/token => 400 Bad Request
[2026-09-14 12:56:23.157][request][INFO] POST /identity/connect/token
[2026-09-14 12:56:23.417][vaultwarden::api::identity][INFO] [redacted email] logged in successfully. IP: 172.18.0.1
[2026-09-14 12:56:23.417][response][INFO] (login) POST /identity/connect/token => 200 OK
[2026-09-14 12:56:24.786][request][INFO] GET /notifications/hub?access_token=eyJ0eXAiOiJKV1QiOiJ...
[2026-09-14 12:56:25.160][request][INFO] GET /api/auth-requests/pending
[2026-09-14 12:56:25.162][response][INFO] (get_auth_requests_pending) GET /api/auth-requests/pending => 200 OK
[2026-09-14 13:09:39.328][request][INFO] GET /api/auth-requests/pending
[2026-09-14 13:09:39.330][response][INFO] (get_auth_requests_pending) GET /api/auth-requests/pending => 200 OK
[2026-09-14 13:09:39.496][request][INFO] POST /identity/connect/token
[2026-09-14 13:09:39.507][response][INFO] (login) POST /identity/connect/token => 200 OK
[2026-09-14 13:10:05.594][request][INFO] GET /api/auth-requests/pending
[2026-09-14 13:10:05.597][response][INFO] (get_auth_requests_pending) GET /api/auth-requests/pending => 200 OK
[2026-09-14 13:10:53.315][request][INFO] GET /api/auth-requests/pending
[2026-09-14 13:10:53.321][response][INFO] (get_auth_requests_pending) GET /api/auth-requests/pending => 200 OK
[2026-09-14 13:14:36.700][request][INFO] GET /api/auth-requests/pending
[2026-09-14 13:14:36.472][response][INFO] (get_auth_requests_pending) GET /api/auth-requests/pending => 200 OK
[NOTICE] You are using a plain text 'ADMIN_TOKEN' which is insecure.
See: https://github.com/dani-garcia/vaultwarden/wiki/Enabling-admin-page#secure-the-admin_token

--- VAULT OPERATIONS ---
[2026-09-13 13:30:34.919][request][INFO] POST /api/ciphers/create
[2026-09-13 13:30:34.940][response][INFO] (post_ciphers_create) POST /api/ciphers/create => 200 OK
[2026-09-13 13:32:37.301][request][INFO] GET /api/ciphers/organization-details?organizationId=91ea5cd-9fe7-4
[2026-09-13 13:32:37.307][response][INFO] (get_org_details) GET /api/ciphers/organization-details?<data..> => 200 OK
[2026-09-13 13:36:48.553][request][INFO] GET /api/ciphers/organization-details?organizationId=91ea5cd-9fe7-4
[2026-09-13 13:36:48.558][response][INFO] (get_org_details) GET /api/ciphers/organization-details?<data..> => 200 OK
[2026-09-13 13:37:23.018][request][INFO] PUT /api/ciphers/3ffa0a30-2eb0-4e48-bf7b-fe8742f1cc9/collections-admin
[2026-09-13 13:37:23.021][response][INFO] (put_collections_admin) PUT /api/ciphers/<cipher_id>/collections-admin => 200 OK
[2026-09-13 13:37:23.021][request][INFO] PUT /api/ciphers/3ffa0a30-2eb0-4e48-bf7b-fe8742f1cc9/collections-admin
[2026-09-13 13:37:23.086][response][INFO] (get_org_details) GET /api/ciphers/organization-details?<data..> => 200 OK
[2026-09-14 12:58:51.700][request][INFO] POST /api/ciphers
[2026-09-14 12:58:51.705][response][INFO] (post_ciphers) POST /api/ciphers => 200 OK
[2026-09-14 13:01:23.470][request][INFO] PUT /api/ciphers/18c6de92-ecdb-480c-a58e-a486690061b0/share
[2026-09-14 13:01:23.480][response][INFO] (put_cipher_share) PUT /api/ciphers/<cipher_id>/share => 200 OK
[2026-09-14 13:02:07.554][request][INFO] PUT /api/ciphers/521ae4e0-439a-483b-aa7a-00dbd025de1e/share
[2026-09-14 13:02:07.563][response][INFO] (put_cipher_share) PUT /api/ciphers/<cipher_id>/share => 200 OK
[2026-09-14 13:02:34.361][request][INFO] POST /api/ciphers/create
[2026-09-14 13:02:34.367][response][INFO] (post_ciphers_create) POST /api/ciphers/create => 200 OK

```

Figure 6.5 — Server log excerpt: the genuine "2FA token not provided" rejection, alongside a successful login event.

6.3 Successful authentication

A successful login was also captured in the logs: "User <account> logged in successfully. IP: 172.18.0.1" (the Docker bridge-network address of the client container/browser host), followed by the expected session activity — a notifications WebSocket connection, pending auth-request polling (used by Bitwarden's "login with device" feature), and a vault sync.

7. RBAC and access control

Vaultwarden implements Bitwarden's organization model: an organization contains one or more collections, vault items are placed into collections, and members are assigned roles and collection access rather than being given blanket account-wide access. This project created a single organization, a default collection, and a second member account to demonstrate the restriction in practice rather than relying on the Admin Console's role labels alone.

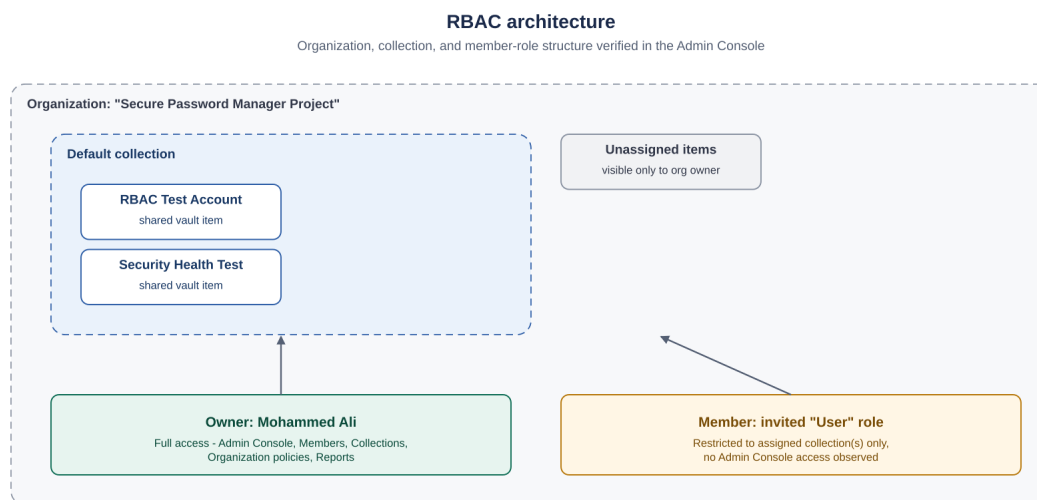


Figure 7.1 — Organization, collection, and member-role structure verified in the Admin Console.

7.1 Organization and collections

An organization named "Secure Password Manager Project" was created, containing a Default collection. Two items - "RBAC Test Account" and "Security Health Test" - were placed into that collection so that access to them could be used as the access-control test signal.

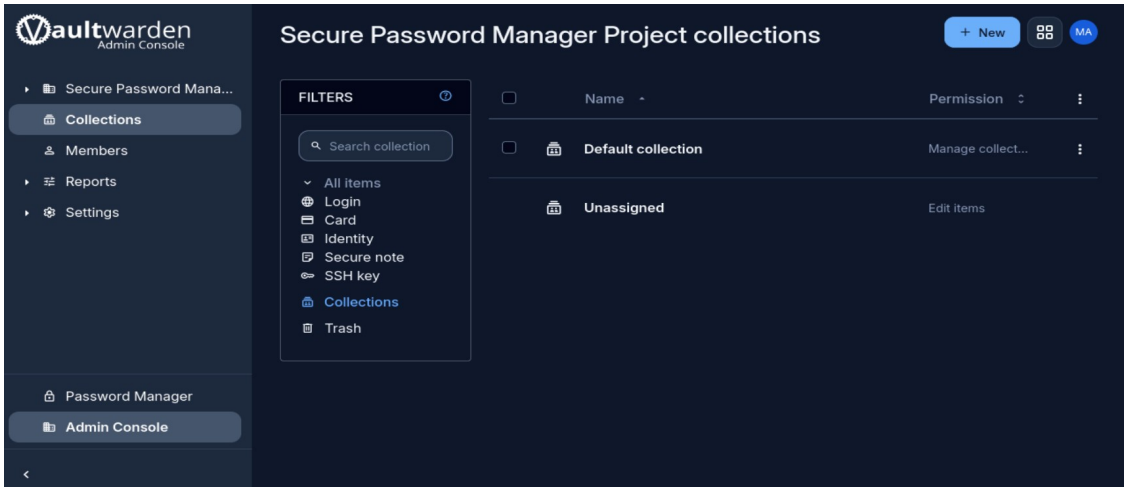


Figure 7.2 — Organization and Default collection created in the Admin Console.

7.2 Member roles

The Members screen shows two accounts: Mohammed Ali as Owner (full administrative access, indicated by the lock icon next to the role), and a second account invited with the "User" role, shown with status "Invited". "User" is Bitwarden's standard non-administrative member role - it can access collections it has been granted, but has no access to organization administration (Members, Policies, or organization-wide Reports).

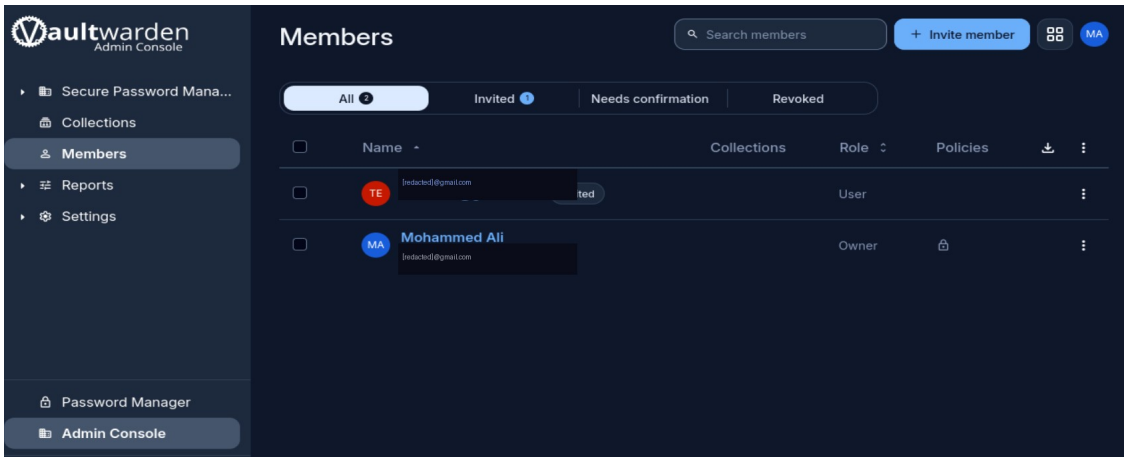


Figure 7.3 — Members list: owner and invited member, with roles (emails redacted).

7.3 Restricted access demonstrated

A session under the second account shows it viewing only the Default collection, seeing the "RBAC Test Account" item, with no Admin Console section available in its left-hand navigation - unlike the owner's session, which includes Admin Console throughout. This is the practical confirmation that the role restriction is enforced at the client-visible level, not just labelled in the members table.

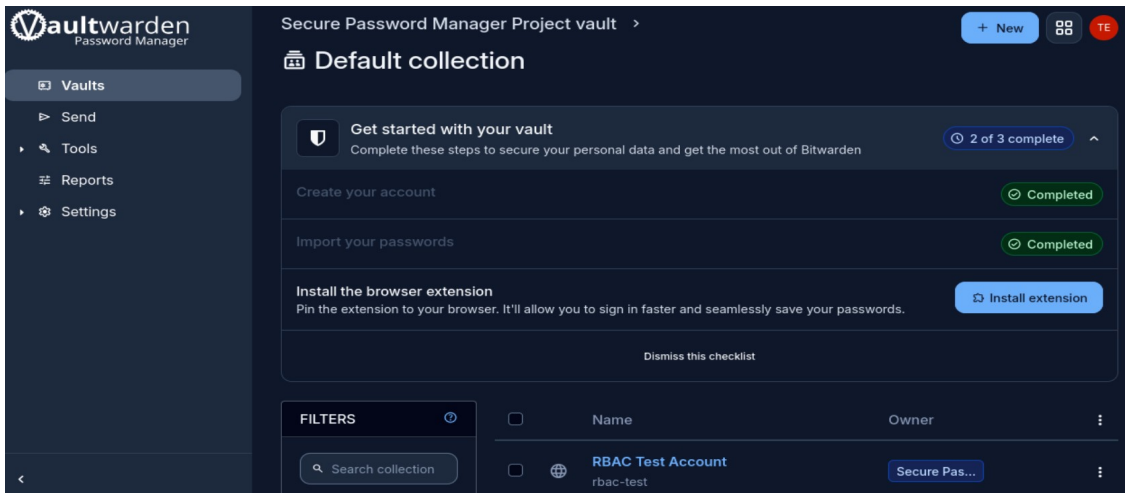


Figure 7.4 — Second account's session, restricted to the Default collection with no Admin Console access.

7.4 Password sharing

Vault-operation log entries captured during testing include two PUT /api/ciphers/{id}/share -> 200 OK calls, which is the API operation Bitwarden clients use to move a personal item into an organization collection (i.e. share it). This confirms sharing occurred, although the corresponding UI screenshot of the share dialog itself was not part of the supplied evidence - the finding rests on the server log rather than a screen capture.

Evidence note

The invited member's status is shown as "Invited" rather than "Confirmed" in the members table screenshot. The separate screenshot of that account actively viewing the shared collection indicates the invitation was accepted at some point in testing, but the two screenshots were not necessarily taken at the same moment, so the exact confirmation step is inferred from the combination of evidence rather than shown directly.

8. Backup and disaster recovery

Two backup strategies were tested: a database-only backup (the SQLite file alone) and a full backup (the database plus the RSA key pair, which is also required to correctly restore organization/sharing functionality). Both were encrypted at rest and both were restored and verified in a separate location before being accepted as successful.

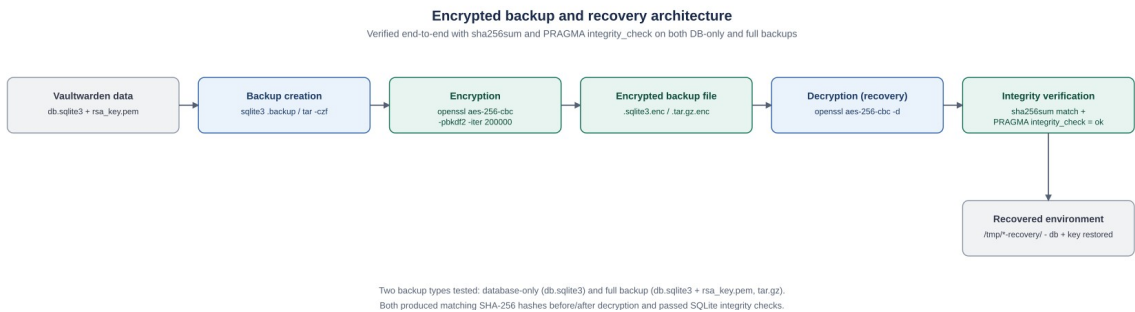


Figure 8.1 — Encrypted backup and recovery architecture, covering both tested backup types.

8.1 Database backup and integrity check

The database backup was created with SQLite's own online backup command, sqlite3 vw-data/db.sqlite3 ".backup ...", rather than a raw file copy, which avoids capturing a database file mid-write. The resulting file (272 KB) passed PRAGMA integrity_check immediately after creation.

```
(ali@kali)-[~/SecurePasswordManagerProject/vaultwarden]
$ sqlite3 vw-data/db.sqlite3 ".backup '/home/ali/SecurePasswordManagerProject/Backups/vaultwarden_db_backup.sqlite3'"

(ali@kali)-[~/SecurePasswordManagerProject/vaultwarden]
$ ls -lh /home/ali/SecurePasswordManagerProject/Backups/vaultwarden_db_backup.sqlite3
-rw-r--r-- 1 ali ali 272K Sep 11 18:04 /home/ali/SecurePasswordManagerProject/Backups/vaultwarden_db_backup.sqlite3

(ali@kali)-[~/SecurePasswordManagerProject/vaultwarden]
$ sqlite3 /home/ali/SecurePasswordManagerProject/Backups/vaultwarden_db_backup.sqlite3 "PRAGMA integrity_check;"
ok
```

Figure 8.2 — SQLite online backup and integrity check.

8.2 Backup encryption

Both the database-only backup and the full backup were encrypted with `openssl enc -aes-256-cbc -salt -pbkdf2 -iter 200000`, a strong symmetric configuration - PBKDF2 with 200,000 iterations substantially exceeds OpenSSL's historical default and mitigates offline brute-force of the backup passphrase. The `file(1)` utility independently confirmed the output as "openssl enc'd data with salted password" rather than merely trusting the file extension.

```
(ali@kali)-[~/SecurePasswordManagerProject/vaultwarden]
$ openssl version
OpenSSL 3.6.3 9 Jun 2026 (Library: OpenSSL 3.6.3 9 Jun 2026)

(ali@kali)-[~/SecurePasswordManagerProject/vaultwarden]
$ openssl enc -aes-256-cbc -salt -pbkdf2 -iter 200000 \
-in /home/ali/SecurePasswordManagerProject/Backups/vaultwarden_db_backup.sqlite3 \
-out /home/ali/SecurePasswordManagerProject/Backups/vaultwarden_db_backup.sqlite3.enc
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:

(ali@kali)-[~/SecurePasswordManagerProject/vaultwarden]
$ ls -lh /home/ali/SecurePasswordManagerProject/Backups/
total 548K
-rw-r--r-- 1 ali ali 272K Sep 11 18:04 vaultwarden_db_backup.sqlite3
-rw-rw-r-- 1 ali ali 273K Sep 11 18:07 vaultwarden_db_backup.sqlite3.enc

(ali@kali)-[~/SecurePasswordManagerProject/vaultwarden]
$ file /home/ali/SecurePasswordManagerProject/Backups/vaultwarden_db_backup.sqlite3.enc
/home/ali/SecurePasswordManagerProject/Backups/vaultwarden_db_backup.sqlite3.enc: openssl enc'd data with salted password

(ali@kali)-[~/SecurePasswordManagerProject/vaultwarden]
$ openssl version
OpenSSL 3.6.3 9 Jun 2026 (Library: OpenSSL 3.6.3 9 Jun 2026)

(ali@kali)-[~/SecurePasswordManagerProject/vaultwarden]
$ ls -lh /home/ali/SecurePasswordManagerProject/Backups/
total 548K
-rw-r--r-- 1 ali ali 272K Sep 11 18:04 vaultwarden_db_backup.sqlite3
-rw-rw-r-- 1 ali ali 273K Sep 11 18:07 vaultwarden_db_backup.sqlite3.enc
```

Figure 8.3 — OpenSSL AES-256-CBC encryption of the database backup, with format confirmed by `file(1)`.

```
(ali@kali)-[~/SecurePasswordManagerProject/vaultwarden]
$ cd ~/SecurePasswordManagerProject

ls -lh Backups/
total 580K
-rw-r--r-- 1 ali ali 272K Sep 11 18:04 vaultwarden_db_backup.sqlite3
-rw-rw-r-- 1 ali ali 273K Sep 11 18:07 vaultwarden_db_backup.sqlite3.enc
-rw-r--r-- 1 ali ali 13K Sep 11 18:15 vaultwarden_full_backup.tar.gz
-rw-rw-r-- 1 ali ali 13K Sep 11 18:16 vaultwarden_full_backup.tar.gz.enc

(ali@kali)-[~/SecurePasswordManagerProject]
$ sqlite3 Backups/vaultwarden_db_backup.sqlite3 "PRAGMA integrity_check;"
ok
```

Figure 8.4 — Backups directory showing both plaintext and encrypted versions of the database and full backups.

8.3 Restore and integrity verification

For the database-only backup, the encrypted file was decrypted to a separate path, and `sha256sum` was used to confirm the decrypted file was byte-identical to the original backup before encryption; `PRAGMA integrity_check` was then run again on the recovered file and returned `ok`.

```

[ali@kali]~/SecurePasswordManagerProject/vaultwarden
$ openssl enc -d -aes-256-cbc -pbkdf2 -iter 200000 \
-in /home/ali/SecurePasswordManagerProject/Backups/vaultwarden_db_backup.sqlite3.enc \
-out /tmp/vaultwarden_db_recovered.sqlite3
enter AES-256-CBC decryption password:

[ali@kali]~/SecurePasswordManagerProject/vaultwarden
$ sha256sum /tmp/vaultwarden_db_recovered.sqlite3
127a41d5e45812971b266e9f63c85434fb3e0d59f2e8c49e5a79b87d5bdfb80 /home/ali/SecurePasswordManagerProject/Backups/vaultwarden_db_backup.sqlite3
127a41d5e45812971b266e9f63c85434fb3e0d59f2e8c49e5a79b87d5bdfb80 /tmp/vaultwarden_db_recovered.sqlite3

[ali@kali]~/SecurePasswordManagerProject/vaultwarden
$ sqlite3 /tmp/vaultwarden_db_recovered.sqlite3 "PRAGMA integrity_check;"
ok

```

Figure 8.5 — Decryption of the database backup with matching SHA-256 hashes and a passing integrity check.

For the full backup, the tar archive (containing db.sqlite3 and rsa_key.pem) was decrypted, extracted to a dedicated recovery directory, and the recovered database again passed its integrity check. The recovered rsa_key.pem was also inspected directly: 1,675 bytes, permissions -rw----- (600, owner-only) in the recovered copy.

```

[ali@kali]~/SecurePasswordManagerProject/vaultwarden
$ ls -lh /home/ali/SecurePasswordManagerProject/Backups/vaultwarden_full_backup.tar.gz*
-rw-r--r-- 1 ali ali 13K Sep 11 18:15 /home/ali/SecurePasswordManagerProject/Backups/vaultwarden_full_backup.tar.gz
-rw-rw-r-- 1 ali ali 13K Sep 11 18:16 /home/ali/SecurePasswordManagerProject/Backups/vaultwarden_full_backup.tar.gz.enc

[ali@kali]~/SecurePasswordManagerProject/vaultwarden
$ mkdir -p /tmp/vaultwarden-recovery

[ali@kali]~/SecurePasswordManagerProject/vaultwarden
$ openssl enc -d -aes-256-cbc -pbkdf2 -iter 200000 \
-in /home/ali/SecurePasswordManagerProject/Backups/vaultwarden_full_backup.tar.gz.enc \
-out /tmp/vaultwarden-recovery/recovered_backup.tar.gz
enter AES-256-CBC decryption password:

[ali@kali]~/SecurePasswordManagerProject/vaultwarden
$ tar -xzf /tmp/vaultwarden-recovery/recovered_backup.tar.gz \
-C /tmp/vaultwarden-recovery/

[ali@kali]~/SecurePasswordManagerProject/vaultwarden
$ ls -lah /tmp/vaultwarden-recovery/
total 292K
drwxrwxr-x 2 ali ali 100 Sep 11 18:18 .
drwxrwxrwt 18 root root 440 Sep 11 18:18 ..
-rw-r--r-- 1 ali ali 272K Sep 11 18:13 db.sqlite3
-rw-rw-r-- 1 ali ali 13K Sep 11 18:18 recovered_backup.tar.gz
-rw----- 1 ali ali 1.7K Sep 11 18:13 rsa_key.pem

[ali@kali]~/SecurePasswordManagerProject/vaultwarden
$ sqlite3 /tmp/vaultwarden-recovery/db.sqlite3 "PRAGMA integrity_check;"
ok

[ali@kali]~/SecurePasswordManagerProject/vaultwarden
$ stat -c '%A %s bytes %n' /tmp/vaultwarden-recovery/rsa_key.pem
-rw----- 1675 bytes /tmp/vaultwarden-recovery/rsa_key.pem

```

Figure 8.6 — Full backup decrypted and extracted to a recovery directory; integrity check passed and rsa_key.pem permissions confirmed.

```

[ali@kali]~/SecurePasswordManagerProject/vaultwarden
$ tar -cf /home/ali/SecurePasswordManagerProject/Backups/vaultwarden_full_backup.tar.gzrm -f /home/ali/SecurePasswordManagerProject/Backups/vaultwarden_full_backup.tar.gz

[ali@kali]~/SecurePasswordManagerProject/vaultwarden
$ sudo tar -czf /home/ali/SecurePasswordManagerProject/Backups/vaultwarden_full_backup.tar.gz \
-C /tmp/vaultwarden-backup db.sqlite3 rsa_key.pem

[ali@kali]~/SecurePasswordManagerProject/vaultwarden
$ tar -tzf /home/ali/SecurePasswordManagerProject/Backups/vaultwarden_full_backup.tar.gz
db.sqlite3
rsa_key.pem

[ali@kali]~/SecurePasswordManagerProject/vaultwarden
$ ls -lh /home/ali/SecurePasswordManagerProject/Backups/vaultwarden_full_backup.tar.gz
-rw-r--r-- 1 root root 13K Sep 11 18:15 /home/ali/SecurePasswordManagerProject/Backups/vaultwarden_full_backup.tar.gz

[ali@kali]~/SecurePasswordManagerProject/vaultwarden
$ sudo chown ali:ali /home/ali/SecurePasswordManagerProject/Backups/vaultwarden_full_backup.tar.gz

[ali@kali]~/SecurePasswordManagerProject/vaultwarden
$ openssl enc -aes-256-cbc -salt -pbkdf2 -iter 200000 \
-in /home/ali/SecurePasswordManagerProject/Backups/vaultwarden_full_backup.tar.gz \
-out /home/ali/SecurePasswordManagerProject/Backups/vaultwarden_full_backup.tar.gz.enc
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:

[ali@kali]~/SecurePasswordManagerProject/vaultwarden
$ file /home/ali/SecurePasswordManagerProject/Backups/vaultwarden_full_backup.tar.gz.enc
/home/ali/SecurePasswordManagerProject/Backups/vaultwarden_full_backup.tar.gz.enc: openssl enc'd data with salted password

[ali@kali]~/SecurePasswordManagerProject/vaultwarden
$ ls -lh /home/ali/SecurePasswordManagerProject/Backups/vaultwarden_full_backup.tar.gz*
-rw-r--r-- 1 ali ali 13K Sep 11 18:15 /home/ali/SecurePasswordManagerProject/Backups/vaultwarden_full_backup.tar.gz
-rw-rw-r-- 1 ali ali 13K Sep 11 18:16 /home/ali/SecurePasswordManagerProject/Backups/vaultwarden_full_backup.tar.gz.enc

```

Figure 8.7 — Creation and encryption of the full backup archive (database plus RSA key).

8.4 Key file permissions - an observed inconsistency

The rsa_key.pem file's permissions were checked at two different points in the evidence, with two different results:

- One check, performed on the host copy under vw-data/, showed -rw-r--r-- (644, world-readable) with owner root:root.

- A separate check (and the recovered copy from the full-backup restore test) showed - rw- - - - - (600, owner-only).

Limitation

Because both readings are present in the evidence and it is not possible to establish their exact order or whether one was corrected into the other, this report does not claim the permission issue was resolved. Both observations are reported, and tightening `rsa_key.pem` to 600 consistently is carried forward as a recommendation in Section 16.

8.5 Recovery limitations

- Recovery was demonstrated to a local `/tmp` directory on the same host, not to a separate machine or a rebuilt environment; a true disaster-recovery drill (rebuilding Vaultwarden from zero using only the backup) was not performed.
- No backup rotation, retention policy, or off-host/offline copy was evidenced; backups observed were all stored alongside the live deployment on the same host.

9. Cross-platform and client integration

Vaultwarden implements the Bitwarden API, so it is architecturally compatible with the full family of official Bitwarden clients: the web vault, browser extensions, desktop applications, and mobile applications. This project tested only the web vault directly; the other clients are documented capabilities of the platform rather than tested features of this deployment, and are reported as such.

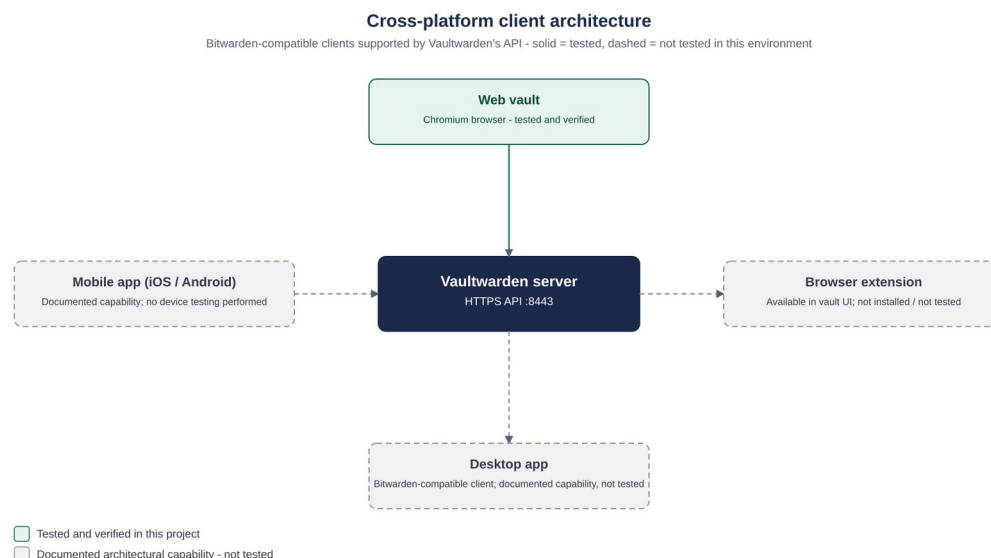


Figure 9.1 — Client architecture: solid/teal indicates tested in this project, dashed/gray indicates a documented capability that was not tested.

9.1 Web vault (tested)

The web vault was accessed via Firefox on the Kali host, first over plain HTTP during initial deployment and subsequently over HTTPS after TLS was configured. Login, TOTP, vault browsing, search, the Admin Console, and organization/collection management were all exercised through this client.

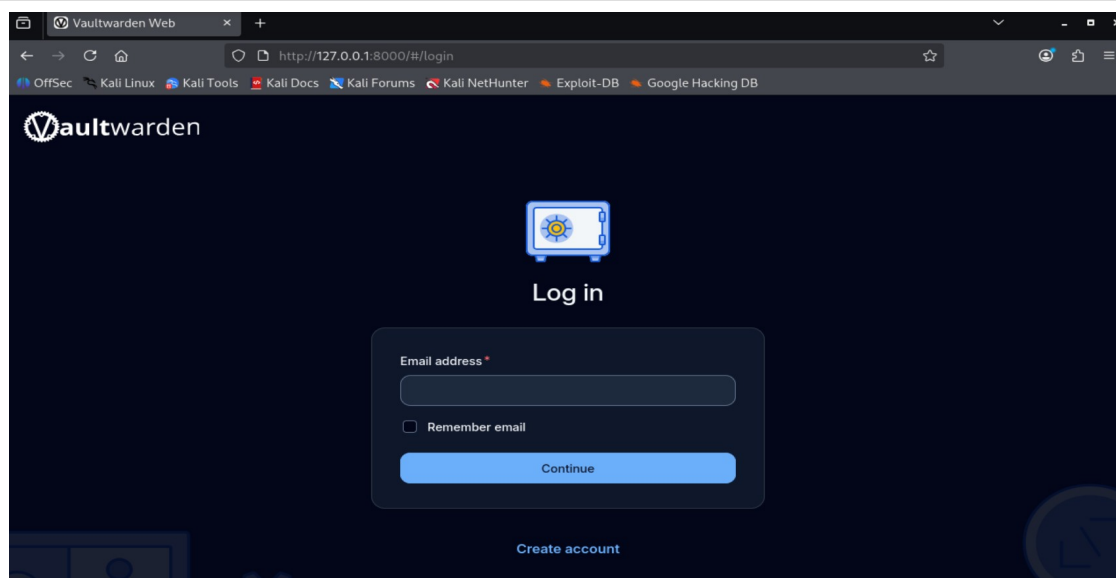


Figure 9.2 — Web vault login page during the initial plain-HTTP deployment stage.

9.2 Browser extension (not tested)

The web vault's onboarding checklist explicitly offers to "Install the browser extension" so that it can "sign in faster and seamlessly save your passwords" - this action was never taken during the project. No autofill or extension-based capture was tested, and this report does not claim otherwise.

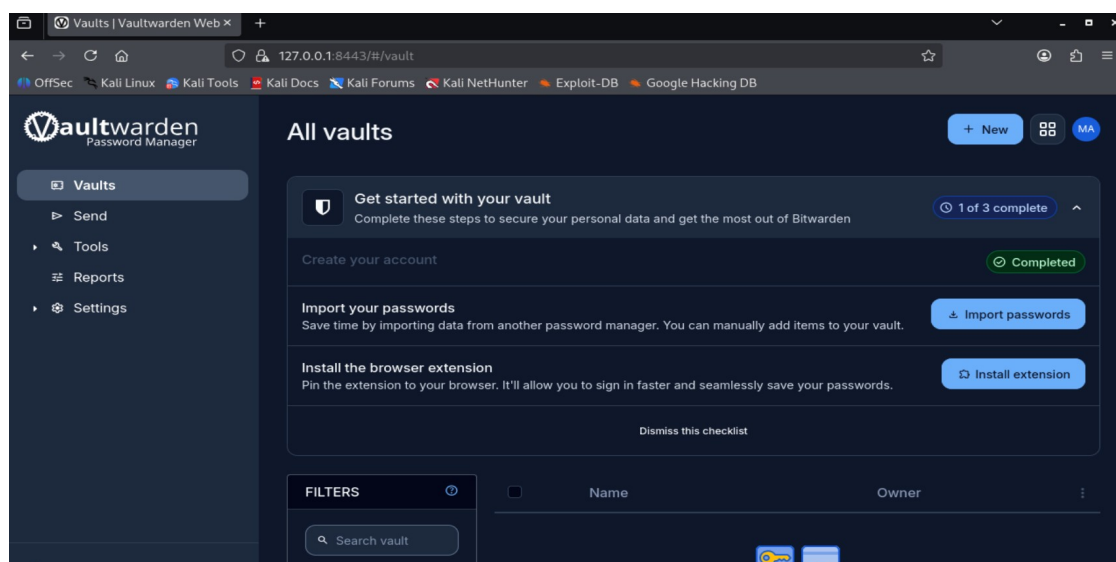


Figure 9.3 — Web vault onboarding checklist showing the browser extension as available but not installed.

9.3 Desktop and mobile applications (not tested)

No desktop or mobile client was installed or connected to this Vaultwarden instance during the project. Bitwarden's published documentation states that these clients work against any compatible server including Vaultwarden, but this is a documented architectural capability, not a verified result of this project.

10. Usability and password security features

Beyond the core cryptographic and access-control requirements, several day-to-day usability features were exercised: vault search, the organization-level weak-password report, and (via server logs) item creation and

sharing. The built-in password generator was not separately screenshotted in use, so its presence is noted as a standard Bitwarden/Vaultwarden feature rather than claimed as directly tested.

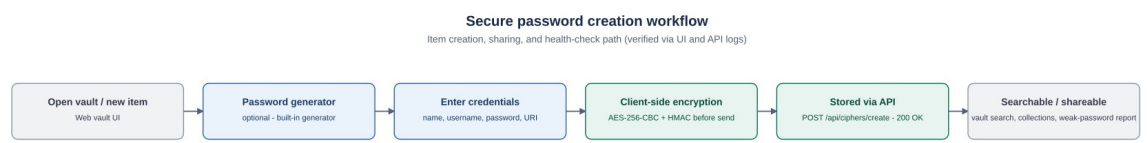


Figure 10.1 — Item creation, sharing, and health-check workflow.

10.1 Vault search

Filtering the Default collection by the term "Health Test" correctly returned only the matching item ("Security Health Test"), confirming the search/filter function operates over item names as expected.

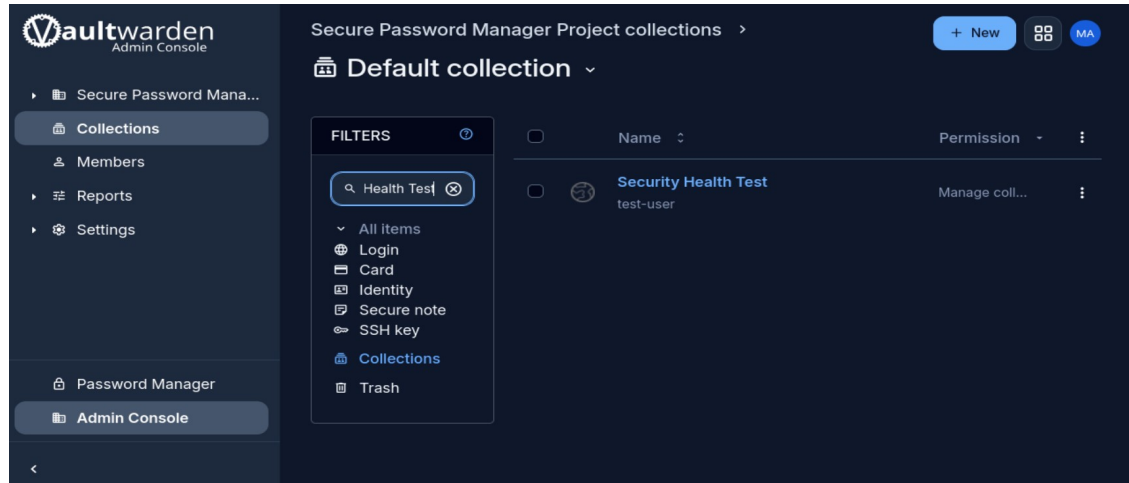


Figure 10.2 — Collection search filtering to a single matching item.

10.2 Password health reporting

The organization Reports > Weak passwords screen identified one item ("Security Health Test") with a password rated "Very Weak", demonstrating that Vaultwarden's password-strength analysis is active and correctly evaluating stored credentials.

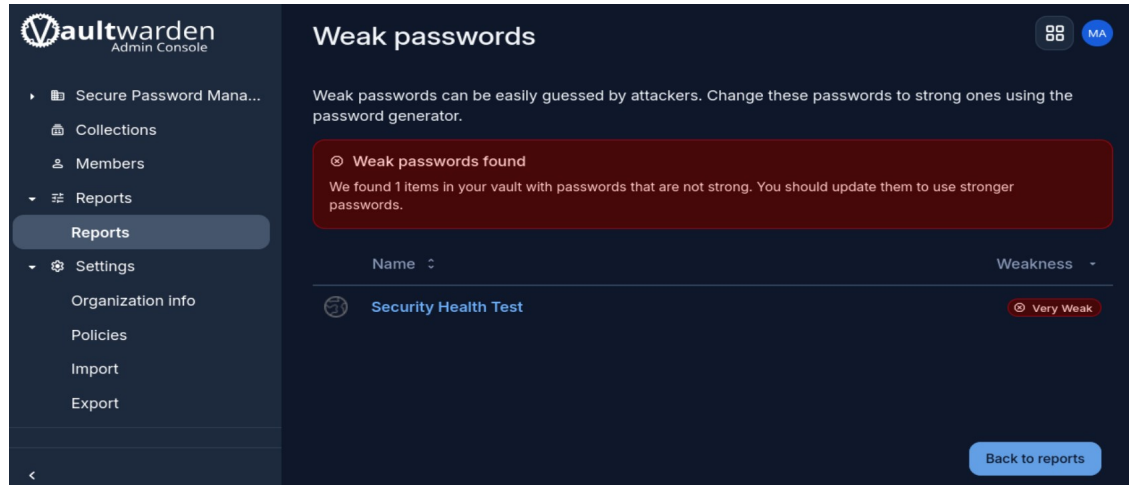


Figure 10.3 — Organization weak-password report flagging one item as "Very Weak".

10.3 Item creation and sharing (via logs)

Vault-operation log entries show multiple `POST /api/ciphers/create -> 200 OK` and `PUT /api/ciphers/{id}/share -> 200 OK` calls, confirming that items were created and subsequently shared into the organization collection over the course of testing, consistent with the items visible in the RBAC and search evidence above.

11. Monitoring and logging

Vaultwarden's Rocket-based web server produces structured, timestamped request/response logs by default, and Docker's own tooling (docker logs, docker inspect, docker stats) provides container-level health and resource monitoring on top of that. No external log-aggregation or alerting system (e.g. a SIEM) was configured; log review in this project was manual, using `grep` against the container's log stream.

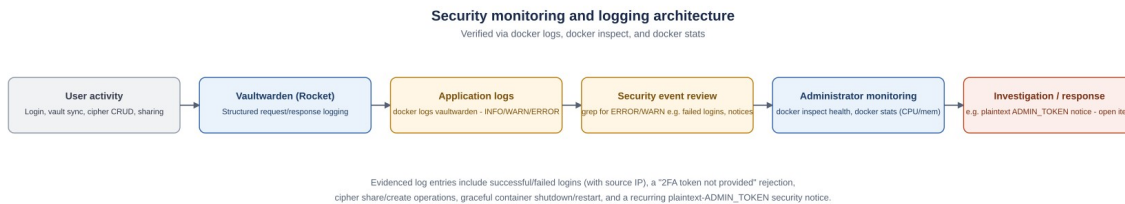


Figure 11.1 — Monitoring and logging architecture, from user activity through to investigation of security-relevant events.

11.1 Application logs

Filtered log excerpts (`grep -iE "POST|PUT|DELETE|GET|ERROR|WARN"`) show a consistent pattern of INFO-level request/response pairs for API calls (cipher CRUD, organization/collection lookups, sync), interspersed with authentication events and, on at least one occasion, an ERROR-level entry for a rejected login.

```

ali@kali: ~/SecurePasswordManagerProject
$ cd ~/SecurePasswordManagerProject/vaultwarden
docker logs --tail 40 vaultwarden 2>&1 | grep -iE "POST|PUT|DELETE|GET|ERROR|WARN" | tail -25
[2026-09-16 08:56:10.224][request][INFO] GET /api/ciphers/381a0137-98ae-4913-97e5-4a998ddddd3/admin
[2026-09-16 08:56:10.227][response][INFO] (get_cipher_admin) GET /api/ciphers/<cipher_id>/admin => 200 OK
[2026-09-16 08:56:12.308][request][INFO] GET /api/organizations/91ea5c2d-9fe7-4bde-9a08-4d3539787ef9/collections/details
[2026-09-16 08:56:12.309][request][INFO] GET /api/ciphers/organization-details?organizationId=91ea5c2d-9fe7-4
[2026-09-16 08:56:12.312][response][INFO] (get_org_details) GET /api/ciphers/organization-details?<data..> => 200 OK
[2026-09-16 08:56:12.312][response][INFO] (get_org_collections_details) GET /api/organizations/<org_id>/collections/details => 200 OK
[2026-09-16 08:57:18.577][request][INFO] GET /api/organizations/91ea5c2d-9fe7-4bde-9a08-4d3539787ef9
[2026-09-16 08:57:18.580][response][INFO] (get_organization) GET /api/organizations/<org_id> => 200 OK
[2026-09-16 08:57:18.580][request][INFO] GET /api/organizations/91ea5c2d-9fe7-4bde-9a08-4d3539787ef9/keys
[2026-09-16 08:57:18.582][response][INFO] (revision_date) GET /api/organizations/<org_id>/keys => 200 OK
[2026-09-16 08:57:48.581][request][INFO] GET /api/accounts/rev
[2026-09-16 08:57:48.610][request][INFO] POST /identity/connect/token
[2026-09-16 08:57:48.625][response][INFO] (login) POST /identity/connect/token => 200 OK
[2026-09-16 08:57:48.655][request][INFO] GET /api/sync?excludeDomains=true
[2026-09-16 08:57:48.660][response][INFO] (sync) GET /api/sync?<data..> => 200 OK
[2026-09-16 08:57:48.994][request][INFO] GET /api/ciphers/organization-details?organizationId=91ea5c2d-9fe7-4
[2026-09-16 08:57:49.000][response][INFO] (get_org_details) GET /api/ciphers/organization-details?<data..> => 200 OK
[2026-09-16 09:05:24.076][request][INFO] POST /identity/accounts/prelogin/password
[2026-09-16 09:05:24.076][request][INFO] GET /api/devices/knowndevice
[2026-09-16 09:05:24.077][response][INFO] (prelogin_password) POST /identity/accounts/prelogin/password => 200 OK
[2026-09-16 09:05:24.077][response][INFO] (get_known_device) GET /api/devices/knowndevice => 200 OK
[2026-09-16 09:06:56.842][request][INFO] POST /identity/connect/token
[2026-09-16 09:06:57.007][vaultwarden::api::identity][ERROR] Username or password is incorrect. Try again. IP: 172.18.0.1. Username: v
[2026-09-16 09:06:57.007][response][INFO] (login) POST /identity/connect/token => 400 Bad Request

```

Figure 11.2 — Filtered docker logs output showing request/response pairs and a failed login event.

11.2 Security-relevant events identified

- A successful login, with account and source IP recorded.
- An ERROR-level "2FA token not provided" rejection (the genuine MFA-failure evidence discussed in Section 6).
- A recurring NOTICE: "You are using a plain text ADMIN_TOKEN which is insecure", with a link to Vaultwarden's own documentation on securing the admin token with an Argon2 hash.
- Cipher create, share, and collection-admin operations, each logged with their endpoint and result code.
- Graceful container shutdown (SIGTERM) and restart, and a Rocket Shield warning that HSTS was auto-enabled because TLS was turned on without an explicit HSTS policy being set.

```

[2026-09-14 12:55:56.942][error][ERROR] 2FA token not provided
[2026-09-14 12:55:56.943][response][INFO] (login) POST /identity/connect/token => 400 Bad Request
[2026-09-14 12:56:23.157][request][INFO] POST /identity/connect/token
[2026-09-14 12:56:23.417][vaultwarden::api::identity][INFO] [redacted email] logged in successfully. IP: 172.18.0.1
[2026-09-14 12:56:23.417][response][INFO] (login) POST /identity/connect/token => 200 OK
[2026-09-14 12:56:24.786][request][INFO] GET /notifications/hub?access_token=keyJ0eXA10IJKV1QiL
[2026-09-14 12:56:25.160][request][INFO] GET /api/auth-requests/pending
[2026-09-14 12:56:25.162][response][INFO] GET /api/auth-requests/pending GET /api/auth-requests/pending => 200 OK
[2026-09-14 13:09:39.328][request][INFO] GET /api/auth-requests/pending
[2026-09-14 13:09:39.330][response][INFO] (get_auth_requests_pending) GET /api/auth-requests/pending => 200 OK
[2026-09-14 13:09:39.496][request][INFO] POST /identity/connect/token
[2026-09-14 13:09:39.507][response][INFO] (login) POST /identity/connect/token => 200 OK
[2026-09-14 13:10:05.594][request][INFO] GET /api/auth-requests/pending
[2026-09-14 13:10:05.597][response][INFO] (get_auth_requests_pending) GET /api/auth-requests/pending => 200 OK
[2026-09-14 13:10:53.245][request][INFO] GET /api/auth-requests/pending
[2026-09-14 13:10:53.321][response][INFO] (get_auth_requests_pending) GET /api/auth-requests/pending => 200 OK
[2026-09-14 13:14:36.470][request][INFO] GET /api/auth-requests/pending
[2026-09-14 13:14:36.472][response][INFO] (get_auth_requests_pending) GET /api/auth-requests/pending => 200 OK
[NOTICE] You are using a plain text "ADMIN_TOKEN" which is insecure.
(see: https://github.com/dani-garcia/vaultwarden/wiki/Enabling-admin-page#secure-the-admin_token)

== VAULT OPERATIONS ==

[2026-09-13 13:30:34.919][request][INFO] POST /api/ciphers/create
[2026-09-13 13:30:34.940][response][INFO] (post_ciphers_create) POST /api/ciphers/create => 200 OK
[2026-09-13 13:32:37.301][request][INFO] GET /api/ciphers/organization-details?organizationId=91eac5cd-9fe7-4
[2026-09-13 13:32:37.307][response][INFO] (get_org_details) GET /api/ciphers/organization-details?<data..> => 200 OK
[2026-09-13 13:36:48.553][request][INFO] GET /api/ciphers/organization-details?organizationId=91eac5cd-9fe7-4
[2026-09-13 13:36:48.558][response][INFO] (get_org_details) GET /api/ciphers/organization-details?<data..> => 200 OK
[2026-09-13 13:37:23.018][request][INFO] PUT /api/ciphers/3fba0a30-2eb0-4e48-bf70-fe847af2fcc9/collections-admin
[2026-09-13 13:37:23.083][request][INFO] GET /api/ciphers/cipher-ids/collections-admin => 200 OK
[2026-09-13 13:37:23.083][request][INFO] GET /api/ciphers/organization-details?organizationId=91eac5cd-9fe7-4
[2026-09-13 13:37:23.086][response][INFO] (get_org_details) GET /api/ciphers/organization-details?<data..> => 200 OK
[2026-09-14 12:58:51.700][request][INFO] POST /api/ciphers
[2026-09-14 12:58:51.705][response][INFO] (post_ciphers) POST /api/ciphers => 200 OK
[2026-09-14 13:01:23.470][request][INFO] PUT /api/ciphers/18cd6e92-ecd6-480c-a58e-a4866990061b0/share
[2026-09-14 13:01:23.480][response][INFO] (put_cipher_share) PUT /api/ciphers/cipher-id/share => 200 OK
[2026-09-14 13:02:07.554][request][INFO] PUT /api/ciphers/521a4e40-439a-483b-a7a-00dd0d25de1e/share
[2026-09-14 13:02:07.557][response][INFO] (put_cipher_share) PUT /api/ciphers/cipher-id/share => 200 OK
[2026-09-14 13:15:34.361][request][INFO] POST /api/ciphers/create
[2026-09-14 13:15:34.367][response][INFO] (post_ciphers_create) POST /api/ciphers/create => 200 OK

```

Figure 11.3 — Authentication events and vault operations, including the plaintext ADMIN TOKEN notice.

```

➔ cd ~/SecurePasswordManagerProject/vaultwarden

echo "=== RECENT VAULTWARDEN LOGS ==="
docker logs --tail 30 vaultwarden

echo
echo "=== CONTAINER RESOURCE STATUS ==="
docker stats vaultwarden --no-stream

=== RECENT VAULTWARDEN LOGS ===
[2026-09-14 13:14:36.472][response][INFO] (get_auth_requests_pending) GET /api/auth-requests/pending => 200 OK
[2026-09-14 13:15:34.361][request][INFO] POST /api/ciphers/create
[2026-09-14 13:15:34.367][response][INFO] (post_ciphers_create) POST /api/ciphers/create => 200 OK
[2026-09-14 13:16:06.931][request][INFO] GET /api/config
[2026-09-14 13:16:06.931][response][INFO] (config) GET /api/config => 200 OK
[2026-09-14 13:24:04.187][request][INFO] GET /api/config
[2026-09-14 13:24:04.187][response][INFO] (config) GET /api/config => 200 OK
[2026-09-14 13:27:07.624][vaultwarden::api:notifications][INFO] Closing WS connection from 172.18.0.1
[2026-09-14 13:27:11.675][vaultwarden][INFO] Received SIGTERM, initiating graceful shutdown
[2026-09-14 13:27:11.675][vaultwarden][INFO] Vaultwarden process exited!

Starting Vaultwarden
Version 1.37.2

This is an *unofficial* Bitwarden implementation, DO NOT use the
official channels to report bugs/features, regardless of client.
Send usage/configuration questions or feature requests to:
  https://github.com/dani-garcia/vaultwarden/discussions or
  https://vaultwarden.discourse.group/
Report suspected bugs/issues in the software itself at:
  https://github.com/dani-garcia/vaultwarden/issues/new

[NOTICE] You are using a plain text 'ADMIN_TOKEN' which is insecure.
Please generate a secure Argon2 PHC string by using 'vaultwarden hash' or 'argon2'.
See: https://github.com/dani-garcia/vaultwarden/wiki/Enabling-admin-page#secure-the-admin_token

[2026-09-14 16:53:33.694][rocket::shield::shield::_][WARN] Detected TLS-enabled liftoff without enabling HSTS.
[2026-09-14 16:53:33.697][rocket::shield::shield::_][WARN] Shield has enabled a default HSTS policy.
[2026-09-14 16:53:33.697][start][INFO] Rocket has launched from https://0.0.0.0:80

=== CONTAINER RESOURCE STATUS ===
CONTAINER ID   NAME          CPU %     MEM USAGE / LIMIT   MEM %     NET I/O     BLOCK I/O  PIDS
c17d5a1df19f   vaultwarden   0.00%     73.23MiB / 6.274GiB  1.14%     6.79kB / 5.16kB  68MB / 77.8kB  13

```

Figure 11.4 — Container restart sequence, Rocket Shield HSTS auto-enable warning, and docker stats resource output.

11.3 The plaintext ADMIN_TOKEN notice was not shown to be resolved

Limitation

A screenshot named "Admin_Token_Remediation_Container_Healthy.png" shows a docker-compose config validation and a container recreate/restart, but does not show the ADMIN_TOKEN value itself being changed to a hashed form. The same plaintext-token NOTICE reappears in a later log capture (Vaultwarden_Monitoring_and_Logs.png, dated after the "remediation" screenshot). On the balance of the evidence, this issue is reported as an open item rather than a completed remediation.

11.4 Container health and resource monitoring

docker inspect confirmed the container's health check reported "healthy" at multiple points across the testing window, and docker stats captured a representative resource snapshot: approximately 73 MiB memory use against a 6.27 GiB limit, negligible CPU, and 13 processes - consistent with light, single-user test traffic rather than production load.

12. Security testing

Twenty test cases were performed or attempted, spanning transport security, cryptography, authentication, access control, backup/recovery, logging, and usability. Each is reported with its objective, procedure, expected result, actual result, evidence file, and security significance. No result below is asserted beyond what its cited evidence shows.

Table 12.1 — Test case summary

ID	Test name	Result
TC-01	HTTPS response security headers	Pass
TC-02	TLS certificate validation	Pass (with caveat)
TC-03	Argon2id KDF configuration	Pass
TC-04	Vault ciphertext format verification	Pass
TC-05	MFA successful login	Pass
TC-06	MFA rejection (missing TOTP token)	Pass
TC-07	Authentication rejection (invalid master password)	Pass
TC-08	RBAC restricted member access	Pass
TC-09	Password sharing	Pass (log-evidenced)
TC-10	Database backup integrity (pre-encryption)	Pass
TC-11	Backup encryption at rest	Pass
TC-12	Database backup restore and hash verification	Pass
TC-13	Full backup (DB + RSA key) restore	Pass
TC-14	Key file permission check	Inconclusive
TC-15	Vault data persistence across restart	Pass
TC-16	Security-relevant application logging	Pass (with open finding)
TC-17	Container health check	Pass
TC-18	Password health / weak-password report	Pass
TC-19	Vault search	Pass
TC-20	Browser-extension compatibility	Not tested

12.1 Detailed test records

TC-01 — HTTPS response security headers

Objective: Confirm the server enforces a modern set of HTTP security headers over TLS.

Procedure: `curl -k -I https://127.0.0.1:8443`

Expected result: 200 OK with HSTS, CSP, X-Frame-Options, X-Content-Type-Options, Permissions-Policy present.

Actual result: All listed headers present, including strict-transport-security: max-age=31536000.

Status: Pass

Evidence: *HTTPS Security Verification.png*; *HTTPS_TLS_Security_Test.png*

Security significance: Reduces clickjacking, MIME-sniffing, and downgrade risk in browser clients.

TC-02 — TLS certificate validation

Objective: Confirm TLS is actually terminated with a certificate, and characterise its trust level.

Procedure: `openssl s_client -connect 127.0.0.1:8443 | openssl x509 -noout -subject -issuer -dates`

Expected result: A valid certificate object with a defined validity window.

Actual result: Self-signed certificate, subject/issuer CN=127.0.0.1, valid Sep 2026-Sep 2027.

Status: Pass (with caveat)

Evidence: *HTTPS_TLS_Security_Test.png*

Security significance: Confirms TLS is active, but a self-signed certificate provides no third-party trust anchor - acceptable for a lab, not for production.

TC-03 — Argon2id KDF configuration

Objective: Confirm key derivation uses a memory-hard, modern algorithm rather than a weak default.

Procedure: Inspect Settings > Security > Keys in the web vault.

Expected result: Argon2id configured with reasonable memory/iteration/parallelism parameters.

Actual result: Argon2id, 32MB memory, 6 iterations, parallelism 4.

Status: Pass

Evidence: *Argon2id Key Derivation Configuration.png*

Security significance: Increases the cost of offline master-password guessing attacks against a stolen database.

TC-04 — Vault ciphertext format verification

Objective: Confirm vault data is stored encrypted, not in plaintext, at the database layer.

Procedure: `sqlite3 db.sqlite3 "SELECT name, data FROM ciphers;"`

Expected result: Encrypted CipherString values, not readable plaintext.

Actual result: All inspected fields (name, data, username, uri) carry the "2." CipherString prefix.

Status: Pass

Evidence: *Cryptographic Configuration Verification.png*; *Encryption Format Verification.png*; *Vaultwarden Encrypted Vault Database Record.png*

Security significance: Confirms field-level encryption is genuinely applied, not merely assumed from configuration.

TC-05 — MFA successful login

Objective: Confirm TOTP is enforced and a valid code allows login.

Procedure: Log in with master password, then supply a valid authenticator-app TOTP code.

Expected result: Login succeeds and is logged.

Actual result: "User <account> logged in successfully" recorded with source IP.

Status: Pass

Evidence: *Security_Relevant_Log_Analysis.png*

Security significance: Confirms the second factor is actually enforced in the login flow, not only configured.

TC-06 — MFA rejection (missing TOTP token)

Objective: Confirm the server rejects login when a required TOTP token is absent.

Procedure: Attempt token exchange without a TOTP code.

Expected result: Login rejected with an authentication error.

Actual result: "2FA token not provided" logged, 400 Bad Request.

Status: Pass

Evidence: *Security_Relevant_Log_Analysis.png*

Security significance: Confirms MFA is enforced server-side, not just presented client-side.

TC-07 — Authentication rejection (invalid master password)

Objective: Confirm the server rejects an incorrect master password.

Procedure: Attempt login with an incorrect password.

Expected result: Login rejected, generic error shown, event logged.

Actual result: "Invalid master password" shown in UI; "Username or password is incorrect" logged with source IP, 400 Bad Request.

Status: Pass

Evidence: *MFA_Failed_Login_Test.png* (content re-classified, see Section 6.2); *Security_Activity_Logs.png*

Security significance: Confirms brute-force attempts against the master password are rejected and logged.

TC-08 — RBAC restricted member access

Objective: Confirm a non-owner member's access is limited to assigned collections.

Procedure: Log in as an invited "User"-role member and inspect available navigation and vault contents.

Expected result: Access limited to the assigned collection; no Admin Console.

Actual result: Second account saw only the Default collection and its items, with no Admin Console entry in navigation.

Status: Pass

Evidence: *RBAC Limited User.png*; *RBAC Member User Role.png*

Security significance: Confirms the principle of least privilege is enforced for organization members, not just configured in role labels.

TC-09 — Password sharing

Objective: Confirm items can be shared into an organization collection.

Procedure: Move a personal item into the organization via the share action; review API logs.

Expected result: Share operation succeeds.

Actual result: PUT /api/ciphers/{id}/share -> 200 OK observed twice in logs.

Status: Pass (log-evidenced)

Evidence: *Security_Relevant_Log_Analysis.png*

Security significance: Confirms the sharing feature functions; no UI screenshot of the share dialog itself was captured.

TC-10 — Database backup integrity (pre-encryption)

Objective: Confirm a database backup is internally consistent immediately after creation.

Procedure: sqlite3 backup file "PRAGMA integrity_check;"

Expected result: Result: ok.

Actual result: ok.

Status: Pass

Evidence: *Backup Database Integrity Verification.png*

Security significance: Confirms the backup mechanism produces a usable, non-corrupt database file.

TC-11 — Backup encryption at rest

Objective: Confirm backup files are encrypted, not stored in plaintext.

Procedure: openssl enc -aes-256-cbc -salt -pbkdf2 -iter 200000; verify with file(1).

Expected result: Encrypted output confirmed by an independent tool.

Actual result: file(1) reports "openssl enc'd data with salted password" for both the DB-only and full backup archives.

Status: Pass

Evidence: *Encrypted Backup Verification.png*; *Full Encrypted Backup Verification.png*

Security significance: Confirms backups do not become a weaker copy of the protected data.

TC-12 — Database backup restore and hash verification

Objective: Confirm a decrypted backup is identical to the original and still valid.

Procedure: Decrypt, then sha256sum original vs. recovered file; re-run integrity_check.

Expected result: Matching hashes; integrity_check = ok.

Actual result: Identical SHA-256 hashes; integrity_check = ok.

Status: Pass

Evidence: *Backup Restore Integrity Test.png*

Security significance: Confirms the backup is actually recoverable, not merely creatable.

TC-13 — Full backup (DB + RSA key) restore

Objective: Confirm the full backup, including the organization RSA key, restores correctly.

Procedure: Decrypt and extract the tar archive; run `integrity_check`; inspect recovered file permissions.

Expected result: Both files recovered; database valid.

Actual result: `db.sqlite3` and `rsa_key.pem` recovered; `integrity_check` = ok; `rsa_key.pem` recovered at 1,675 bytes, mode 600.

Status: Pass

Evidence: *Full Backup Recovery Test.png*

Security significance: Confirms full-environment recovery, including the key material needed for organization features, is possible.

TC-14 — Key file permission check

Objective: Confirm `rsa_key.pem` is not world-readable.

Procedure: `stat -c '%A %U:%G %n' rsa_key.pem`, checked at two points in testing.

Expected result: Owner-only permissions (600).

Actual result: Inconsistent: 600 in one check, 644 in another (see Section 8.4).

Status: Inconclusive

Evidence: *Secure Key File Permissions.png*; *Vaultwarden Persistent Storage Verification.png*

Security significance: A world-readable private key on a multi-user host would allow any local user to read it; the final state could not be confirmed from the evidence.

TC-15 — Vault data persistence across restart

Objective: Confirm vault data survives container restarts.

Procedure: Inspect `vw-data/db.sqlite3` on the host and via `docker exec` after multiple restarts across several days.

Expected result: Consistent, growing database file across sessions.

Actual result: Database present and consistent across checks spanning 10-16 Sep.

Status: Pass

Evidence: *Vaultwarden Persistent Storage Verification.png*; *Vaultwarden Persistent Storage.png*

Security significance: Confirms the bind-mount persistence strategy works as intended.

TC-16 — Security-relevant application logging

Objective: Confirm authentication and vault operations are logged with enough detail for review.

Procedure: `docker logs vaultwarden, filtered for POST/PUT/DELETE/GET/ERROR/WARN.`

Expected result: Structured, timestamped entries covering auth and vault operations.

Actual result: Confirmed, including source IPs on auth events and a persistent plaintext-ADMIN_TOKEN notice.

Status: Pass (with open finding)

Evidence: *Security_Activity_Logs.png; Security_Relevant_Log_Analysis.png; Vaultwarden_Monitoring_and_Logs.png*

Security significance: Enables after-the-fact security review; the ADMIN_TOKEN notice itself is a separate open finding (Section 11.3).

TC-17 — Container health check

Objective: Confirm the container reports and maintains a healthy state.

Procedure: `docker inspect --format '{{.State.Health.Status}}'`.

Expected result: "healthy".

Actual result: "healthy" at every check performed.

Status: Pass

Evidence: *Vaultwarden_Health_and_HTTPS.png; Vaultwarden Environment Verification.png*

Security significance: Confirms basic service availability was maintained across the testing window.

TC-18 — Password health / weak-password report

Objective: Confirm the built-in password-strength analysis is functioning.

Procedure: Open Organization Reports > Weak passwords.

Expected result: Weak items correctly identified.

Actual result: One item correctly flagged "Very Weak".

Status: Pass

Evidence: *Password_Health_Report.png*

Security significance: Confirms a genuinely useful security-hygiene feature is active for organization members.

TC-19 — Vault search

Objective: Confirm search/filter returns correct results.

Procedure: Filter the Default collection by a partial item name.

Expected result: Only matching items returned.

Actual result: Filtering by "Health Test" returned exactly the matching item.

Status: Pass

Evidence: *Vault_Search_and_Organization.png*

Security significance: Basic usability confirmation; not a security control in itself.

TC-20 — Browser-extension compatibility

Objective: Confirm autofill/extension integration functions.

Procedure: Not performed - extension never installed.

Expected result: N/A.

Actual result: Not tested.

Status: Not tested

Evidence: *Admin Vault Dashboard.png* (shows extension offered, not installed)

Security significance: Cannot be claimed as working; reported honestly as an untested capability.

13. OWASP / security control mapping

The table below maps implemented controls to relevant OWASP guidance (principally the Password Storage Cheat Sheet, the Authentication Cheat Sheet, and the Access Control Cheat Sheet). Where a control is a recommended best practice rather than something implemented in this project, it is marked as such rather than implied to be complete.

Table 13.1 — OWASP-aligned control mapping

Security principle	Implementation	Evidence	Status
Use a memory-hard password hashing / KDF algorithm	Argon2id, 32MB / 6 iter / parallelism 4	Argon2id Key Derivation Configuration.png	Implemented
Encrypt sensitive data at rest	AES-256-CBC + HMAC-SHA256 per field (CipherString)	Encryption Format Verification.png	Implemented
Enforce multi-factor authentication	TOTP two-step login	MFA TOTP Enabled.png; auth logs	Implemented
Fail authentication securely and log failures	Generic error messages; failures logged with source IP	Security_Activity_Logs.png	Implemented
Use TLS for all traffic; enable HSTS	Rocket TLS + auto-enabled HSTS (max-age=31536000)	HTTPS Security Verification.png	Implemented (self-signed cert)
Use a trusted CA-issued certificate in production	Not applicable to this lab	HTTPS_TLS_Security_Test.png (self-signed)	Recommended, not implemented
Principle of least privilege / RBAC	Organization roles + collection-scoped access	RBAC Limited User.png	Implemented
Secure secrets management (e.g. admin credentials)	ADMIN_TOKEN should be an Argon2 hash, not plaintext	Vaultwarden NOTICE in logs	Not implemented (open finding)
Disable unnecessary open registration	SIGNUPS_ALLOWED should be false outside onboarding	docker-compose config	Recommended, not implemented
Encrypt and test backups	AES-256-CBC/PBKDF2 backups, restore-tested	Backup Restore Integrity Test.png	Implemented
Protect private key material with restrictive file permissions	rsa_key.pem should be 600 consistently	Conflicting 644 / 600 readings	Partially implemented / inconsistent
Log security-relevant events	Structured request/response + auth-event logs	Security_Relevant_Log_Analysis.png	Implemented
Centralised log review / alerting	Manual grep-based review only	N/A	Recommended, not implemented

14. Threat model and risk analysis

The threat model below is organised around three trust zones observed in the deployment: the untrusted network the client connects over, the Docker host boundary where TLS is terminated and authentication/authorization occur, and the vaultwarden container's data-at-rest layer. For each threat, the existing control and residual risk are stated as they were actually observed in the evidence - no threat below has been invented beyond what the deployment's own architecture and logs make relevant.

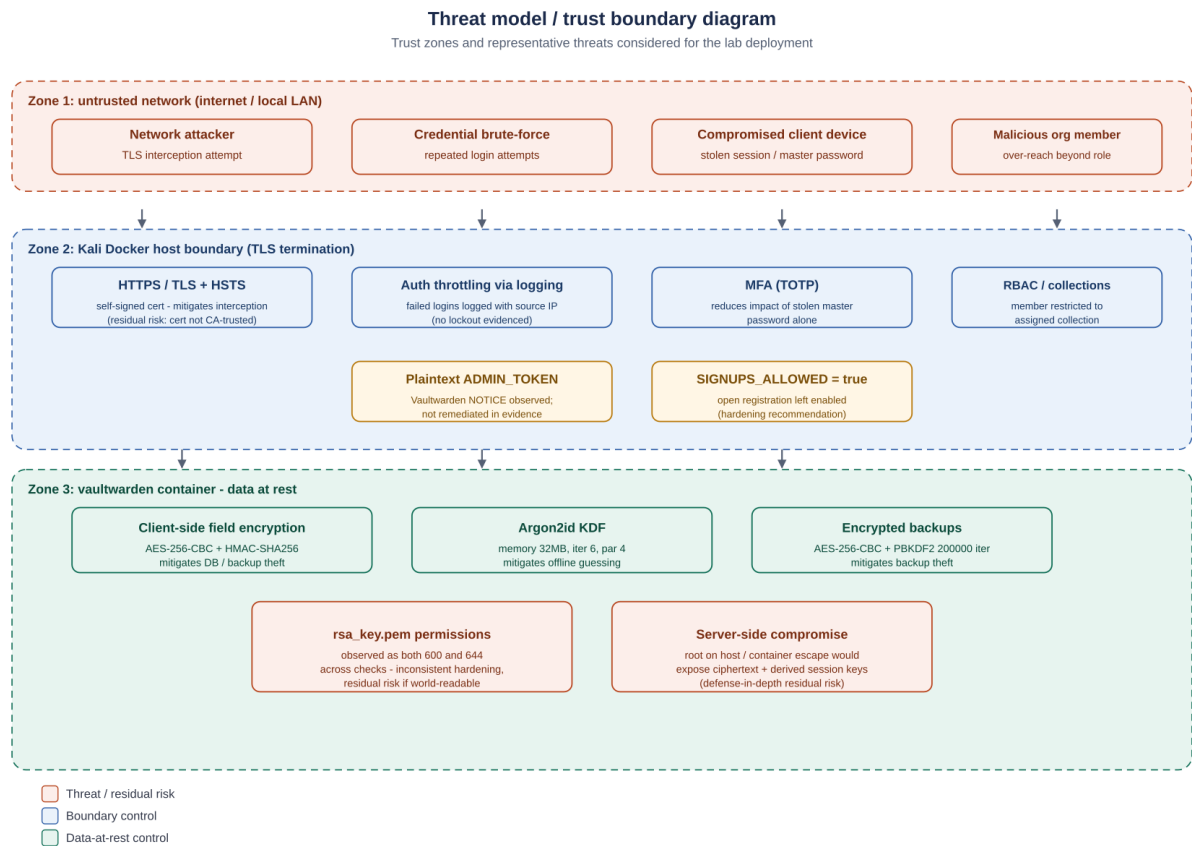


Figure 14.1 — Trust zones, controls, and residual risks identified from the deployment evidence.

Table 14.1 — Threat model and risk register

Threat	Attack surface	Potential impact	Existing control	Residual risk	Recommended mitigation
Network interception (MITM)	Client-server TLS channel	Credential/session disclosure	HTTPS + HSTS	Medium - certificate not CA-trusted, so clients must bypass a warning or pin the cert	Issue a certificate from a trusted CA or internal PKI before any non-lab use
Credential brute-force	Login endpoint	Account takeover	Failed logins logged with source IP	Medium - no evidence of rate-limiting or account lockout	Add rate-limiting / fail2ban-style blocking on repeated auth failures
MFA bypass via stolen device or SIM-based attack (Duo/SMS-style)	Two-step login	Account takeover despite MFA	TOTP app-based (not SMS); Duo available but unused	Low - TOTP avoids SIM-swap risk, but device loss is not separately mitigated (recovery code exists but was not tested)	Test and document the recovery-code path; consider a second MFA method for the owner account

Threat	Attack surface	Potential impact	Existing control	Residual risk	Recommended mitigation
Compromised admin token	Vaultwarden admin panel	Full administrative compromise (all organizations/users)	None confirmed - token stored in plain text per Vaultwarden's own NOTICE	High - plaintext token is readable by anyone with host/container file access	Convert ADMIN_TOKEN to an Argon2 PHC hash as Vaultwarden's own documentation instructs
Database or backup theft	Host filesystem / backup files	Exposure of vault ciphertext	AES-256 field encryption; AES-256-CBC/PBKDF2 backup encryption	Low - ciphertext requires the master password to decrypt	Store backups off-host / offline; add backup rotation and retention policy
Private key exposure (rsa_key.pem)	Host filesystem	Compromise of organization/sharing key material	Permissions observed as 600 in one check	Medium - a separate check showed 644 (world-readable); final state unconfirmed	Explicitly set and verify 600 permissions as part of deployment automation
Unauthorized organization access / privilege escalation	Organization membership	Access to shared vault items beyond intended role	RBAC with collection-scoped "User" role	Low - restricted access demonstrated directly	Periodically audit organization membership and collection assignments
Malicious or careless insider (org member)	Any shared collection	Data exfiltration from shared items	Collection-level scoping limits blast radius	Medium - a User-role member can still read/export items in collections they are granted	Apply least-privilege collection assignment; enable/review export restrictions via organization policies
Open self-registration abuse	Signup endpoint	Unauthorized account creation if exposed beyond the lab	None - SIGNUPS_ALLOWED=true	Medium if ever exposed beyond loopback/local network	Set SIGNUPS_ALLOWED=false and invite users explicitly in any non-lab deployment

15. Limitations

The limitations below are grouped as laboratory constraints (deliberate or practical boundaries of the project's scope) and as open findings (issues the evidence itself surfaced but did not show resolved). Neither category is presented as a failure of the project - both are exactly the kind of honest accounting a controlled academic exercise should produce.

15.1 Laboratory constraints (by design)

- Self-signed TLS certificate: appropriate for a local lab, not for a production trust model.
- Loopback / local-network-only deployment: no public exposure, reverse proxy, or DNS hostname was used.
- No mobile client (iOS/Android) testing was performed.
- The browser extension was never installed, so autofill/extension integration was not tested.
- No dedicated recovery-code, emergency-access, SSO, or Duo testing was performed - these remain documented architectural capabilities of Vaultwarden, not verified results of this project.
- No load, performance, or multi-user concurrency testing was performed.
- Vaultwarden is an unofficial, community-maintained Bitwarden-compatible server; some features of the official Bitwarden ecosystem may not be available or may behave differently.

15.2 Open findings (surfaced by the evidence, not shown resolved)

- Plaintext ADMIN_TOKEN: Vaultwarden's own log NOTICE flags this as insecure; no evidence confirms it was converted to a hashed value, and the notice recurs in a later log capture.
- rsa_key.pem permissions: observed as both 644 (world-readable) and 600 (owner-only) at different points; the final, current state cannot be confirmed from the available evidence.
- SIGNUPS_ALLOWED remains true in the final configuration, which is a reasonable default for a single-operator lab but not for anything beyond it.
- No rate-limiting or account-lockout mechanism was evidenced for repeated failed logins.

16. Security recommendations

The following are recommendations for moving this deployment from a laboratory proof of concept towards a production-suitable configuration. They are explicitly recommendations, not completed project features, and several map directly to the open findings and residual risks identified in Sections 14 and 15.

Immediate hardening (directly evidenced issues)

- Convert ADMIN_TOKEN to an Argon2 PHC hash as instructed by Vaultwarden's own log notice.
- Explicitly set and verify rsa_key.pem at 600 permissions as part of the deployment process, rather than relying on defaults.
- Set SIGNUPS_ALLOWED=false once all intended accounts have been created.

Transport and network

- Replace the self-signed certificate with one from a trusted CA (public or internal PKI) before any use beyond the local lab.
- Deploy behind a reverse proxy with a proper DNS hostname if the service is ever exposed beyond loopback/local network.
- Apply network segmentation / a host firewall restricting access to the Vaultwarden port.

Authentication and monitoring

- Add rate-limiting or automated blocking (e.g. fail2ban) against repeated failed-login attempts.
- Forward logs to a centralized logging system with alerting on authentication failures and admin-panel access.
- Test the TOTP recovery-code path and document it for account-recovery scenarios.

Backup and operations

- Establish a backup rotation and retention policy rather than ad-hoc backup files.
- Store at least one backup copy off-host or offline.
- Perform a full disaster-recovery drill (rebuilding the environment from backup alone) rather than restoring only to a temporary directory on the same host.
- Apply routine security patching to the host OS, Docker engine, and the Vaultwarden image.

Governance

- Periodically audit organization membership, roles, and collection assignments.
- Define an incident-response process for suspected credential or admin-token compromise.

- Document device-security expectations for anyone accessing the vault (e.g. session timeout, device trust).

17. Conclusion

This project deployed Vaultwarden in a controlled Kali Linux / Docker environment and applied a defined set of security controls covering transport encryption, vault-level cryptography, multi-factor authentication, role-based access control, encrypted backups, and application logging. Rather than taking the presence of a configuration setting as proof of security, each control was tested directly against the running system: HTTP responses were inspected with curl, ciphertext was queried directly from the SQLite database, backups were decrypted and hash-verified, and a second account was used to demonstrate access restriction in practice.

What was implemented and verified: HTTPS/TLS with a full security-header set; Argon2id key derivation; AES-256-CBC/HMAC-SHA256 field-level vault encryption; TOTP multi-factor authentication with both a successful login and two distinct, correctly-logged failure paths; an organization/collection RBAC structure with a demonstrated access restriction; AES-256-CBC/PBKDF2-encrypted backups with a tested and hash-verified restore path for both database-only and full backups; and structured application logging covering authentication and vault operations.

What was partially verified or left as an open finding: the `rsa_key.pem` permission state, observed inconsistently across two checks; the plaintext `ADMIN_TOKEN`, flagged by Vaultwarden itself but not shown to be remediated; and password sharing, confirmed via server logs rather than a UI walkthrough.

What remained out of scope or untested: mobile client and browser-extension testing, SSO, Duo, emergency access, and Bitwarden Send were not exercised, and no load, penetration, or multi-host disaster-recovery testing was performed.

The project does not claim 100% completion of every conceivable password-manager security requirement, and it does not present a self-hosted, self-signed, single-operator lab deployment as production-ready. What it does show is that the core security controls a password manager needs - strong key derivation, genuine field-level encryption, enforced multi-factor authentication, working access control, and recoverable encrypted backups - were correctly configured and independently verified, while the specific gaps that remain (an unhashed admin token, inconsistent key-file permissions, and the absence of rate-limiting) are identified precisely enough to be closed before any production use. The main lesson carried forward from this exercise is that configuration and verification are two different activities, and that a security claim is only as strong as the test that was actually run to support it.

18. References

Vaultwarden - official repository and documentation - <https://github.com/dani-garcia/vaultwarden>

Vaultwarden Wiki - Enabling the admin page / securing the admin token - <https://github.com/dani-garcia/vaultwarden/wiki/Enabling-admin-page>

Bitwarden - official documentation - <https://bitwarden.com/help/>

OWASP Password Storage Cheat Sheet - https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

OWASP Authentication Cheat Sheet - https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

OWASP Access Control Cheat Sheet - https://cheatsheetseries.owasp.org/cheatsheets/Access_Control_Cheat_Sheet.html

Docker - official documentation - <https://docs.docker.com/>

Docker Compose - official documentation - <https://docs.docker.com/compose/>

SQLite documentation - <https://www.sqlite.org/docs.html>

OpenSSL documentation - <https://docs.openssl.org/>

Kali Linux documentation - <https://www.kali.org/docs/>

Appendices

Appendix A — Evidence index (requirement to evidence mapping)

This index maps each project requirement to the report section and the underlying evidence file(s) that support it, for quick cross-checking against the original screenshot set.

Table A.1 — Requirement-to-evidence index

Requirement	Report section	Key evidence file(s)
Environment setup	2	Docker Installation Verification.png; Kali Linux Environment.png
HTTPS / TLS	4, 6	HTTPS Security Verification.png; HTTPS_TLS_Security_Test.png
Encryption / KDF	5	Argon2id Key Derivation Configuration.png; Encryption Format Verification.png
Authentication / MFA	6	MFA TOTP Enabled.png; Security_Relevant_Log_Analysis.png
RBAC	7	Organization Created.png; RBAC Limited User.png; RBAC Member User Role.png
Backup / recovery	8	Backup Database Integrity Verification.png; Full Backup Recovery Test.png
Cross-platform clients	9	Vaultwarden Web Login Page.png; Admin Vault Dashboard.png
Usability / password health	10	Vault_Search_and_Organization.png; Password_Health_Report.png
Monitoring / logging	11	Security_Activity_Logs.png; Vaultwarden_Monitoring_and_Logs.png
Security testing	12	See Appendix C

Appendix B — Configuration evidence

Final hardened Docker Compose configuration, as returned by docker-compose config:

```
name: vaultwarden
services:
  vaultwarden:
    container_name: vaultwarden
    environment:
      DOMAIN: https://127.0.0.1:8443
      ROCKET_TLS: '{certs="/ssl/cert.pem",key="/ssl/key.pem"}'
      SIGNUPS_ALLOWED: "true"
    image: vaultwarden/server:latest
    ports:
      - mode: ingress
        target: 80
        published: "8443"
        protocol: tcp
    restart: unless-stopped
    volumes:
      - type: bind
        source: /home/ali/SecurePasswordManagerProject/vaultwarden/vw-data
        target: /data
      - type: bind
```

```
source: /home/ali/SecurePasswordManagerProject/vaultwarden/tls
target: /ssl
read_only: true
```

Verified component versions: Vaultwarden 1.37.2 (Web-Vault 2026.7.0); Docker 28.5.2; Docker Compose 2.40.3; SQLite 3.46.1; OpenSSL 3.6.3; Kali GNU/Linux Rolling 2026.3.

Appendix C — Security test results (compact)

Table C.1 — Full test result set with evidence references

ID	Test name	Result	Evidence file(s)
TC-01	HTTPS response security headers	Pass	HTTPS Security Verification.png; HTTPS_TLS_Security_Test.png
TC-02	TLS certificate validation	Pass (with caveat)	HTTPS_TLS_Security_Test.png
TC-03	Argon2id KDF configuration	Pass	Argon2id Key Derivation Configuration.png
TC-04	Vault ciphertext format verification	Pass	Cryptographic Configuration Verification.png; Encryption Format Verification.png; Vaultwarden Encrypted Vault Database Record.png
TC-05	MFA successful login	Pass	Security_Relevant_Log_Analysis.png
TC-06	MFA rejection (missing TOTP token)	Pass	Security_Relevant_Log_Analysis.png
TC-07	Authentication rejection (invalid master password)	Pass	MFA_Failed_Login_Test.png (content re-classified, see Section 6.2); Security_Activity_Logs.png
TC-08	RBAC restricted member access	Pass	RBAC Limited User.png; RBAC Member User Role.png
TC-09	Password sharing	Pass (log-evidenced)	Security_Relevant_Log_Analysis.png
TC-10	Database backup integrity (pre-encryption)	Pass	Backup Database Integrity Verification.png
TC-11	Backup encryption at rest	Pass	Encrypted Backup Verification.png; Full Encrypted Backup Verification.png
TC-12	Database backup restore and hash verification	Pass	Backup Restore Integrity Test.png
TC-13	Full backup (DB + RSA key) restore	Pass	Full Backup Recovery Test.png
TC-14	Key file permission check	Inconclusive	Secure Key File Permissions.png; Vaultwarden Persistent Storage Verification.png
TC-15	Vault data persistence across restart	Pass	Vaultwarden Persistent Storage Verification.png; Vaultwarden Persistent Storage.png
TC-16	Security-relevant application logging	Pass (with open finding)	Security_Activity_Logs.png; Security_Relevant_Log_Analysis.png; Vaultwarden_Monitoring_and_Logs.png
TC-17	Container health check	Pass	Vaultwarden_Health_and_HTTPS.png; Vaultwarden Environment Verification.png
TC-18	Password health / weak-password report	Pass	Password_Health_Report.png
TC-19	Vault search	Pass	Vault_Search_and_Organization.png
TC-20	Browser-extension compatibility	Not tested	Admin Vault Dashboard.png (shows extension offered, not installed)

Appendix D — Backup and recovery evidence

Table D.1 — Backup artifacts and their verification results

Artifact	Size	Verification
vaultwarden_db_backup.sqlite3	272 KB	PRAGMA integrity_check = ok
vaultwarden_db_backup.sqlite3.enc	273 KB	file(1): openssl enc'd, salted; decrypt -> sha256sum match
vaultwarden_full_backup.tar.gz	13 KB	tar -tzf lists db.sqlite3 + rsa_key.pem
vaultwarden_full_backup.tar.gz.enc	13 KB	file(1): openssl enc'd, salted; decrypt + extract -> integrity_check = ok

Appendix E — Screenshot evidence set (44 files)

The complete set of screenshots supplied for this project, listed by filename. Detailed descriptions and cross-references appear throughout Sections 2-12; this list is provided for completeness and audit.

- 1. Admin Vault Dashboard.png
- 2. Admin_Token_Remediation_Container_Healthy.png
- 3. Argon2id Key Derivation Configuration.png
- 4. Backup Database Integrity Verification.png
- 5. Backup Restore Integrity Test.png
- 6. Backup_Integrity_Test.png
- 7. Compose Configuration Validation.png
- 8. Cryptographic Configuration Verification.png
- 9. Docker Installation Verification.png
- 10. Docker User Access.png
- 11. Encrypted Backup Verification.png
- 12. Encryption Format Verification.png
- 13. Full Backup Recovery Test.png
- 14. Full Encrypted Backup Verification.png
- 15. HTTPS Security Verification.png
- 16. HTTPS_TLS_Security_Test.png
- 17. KDF Database Configuration Verification.png
- 18. Kali Linux Environment.png
- 19. Kali Network Configuration.png
- 20. MFA Login Verification.png
- 21. MFA TOTP Enabled.png
- 22. MFA_Failed_Login_Test.png
- 23. Organization Created.png
- 24. Password_Health_Report.png
- 25. RBAC Limited User.png
- 26. RBAC Member User Role.png
- 27. Secure Key File Permissions.png
- 28. Security_Activity_Logs.png

- 29. Security_Relevant_Log_Analysis.png
- 30. Vault_Search_and_Organization.png
- 31. Vaultwarden Container Running.png
- 32. Vaultwarden Docker Compose Configuration.png
- 33. Vaultwarden Docker Image.png
- 34. Vaultwarden Encrypted Vault Database Record.png
- 35. Vaultwarden Encrypted Vault DatabaseSchema.png
- 36. Vaultwarden Environment Verification.png
- 37. Vaultwarden Persistent Storage Verification.png
- 38. Vaultwarden Persistent Storage.png
- 39. Vaultwarden Security Architecture Verification.png
- 40. Vaultwarden Security Configuration.png
- 41. Vaultwarden Startup Logs.png
- 42. Vaultwarden Web Login Page.png
- 43. Vaultwarden_Health_and_HTTPS.png
- 44. Vaultwarden_Monitoring_and_Logs.png

Appendix F — Diagram package

All nine required diagrams are embedded at their point of relevance in the body of this report (Sections 4-8, 9-11, and 14). They are also supplied as a standalone high-resolution PNG/SVG package alongside this report:

- 01_architecture — Overall security architecture (Section 4)
- 02_encryption — Encryption and key-management flow (Section 5)
- 03_auth_mfa — Authentication and MFA flow (Section 6)
- 04_rbac — RBAC architecture (Section 7)
- 05_backup — Encrypted backup and recovery architecture (Section 8)
- 06_crossplatform — Cross-platform client architecture (Section 9)
- 07_monitoring — Security monitoring and logging architecture (Section 11)
- 08_password_workflow — Secure password creation workflow (Section 10)
- 09_threat_model — Threat model / trust boundary diagram (Section 14)